



UNSALINK
STUDIOS

DBMS

PRIMER EXAMEN PARCIAL



Por Alex Cañapataña Vargas
Alexander Huayhua Perez

CONTENIDOS

01 Introducción

02 Sobre la globalización

03 Propuestas generales

04 Las principales ideas

05 Datos generales

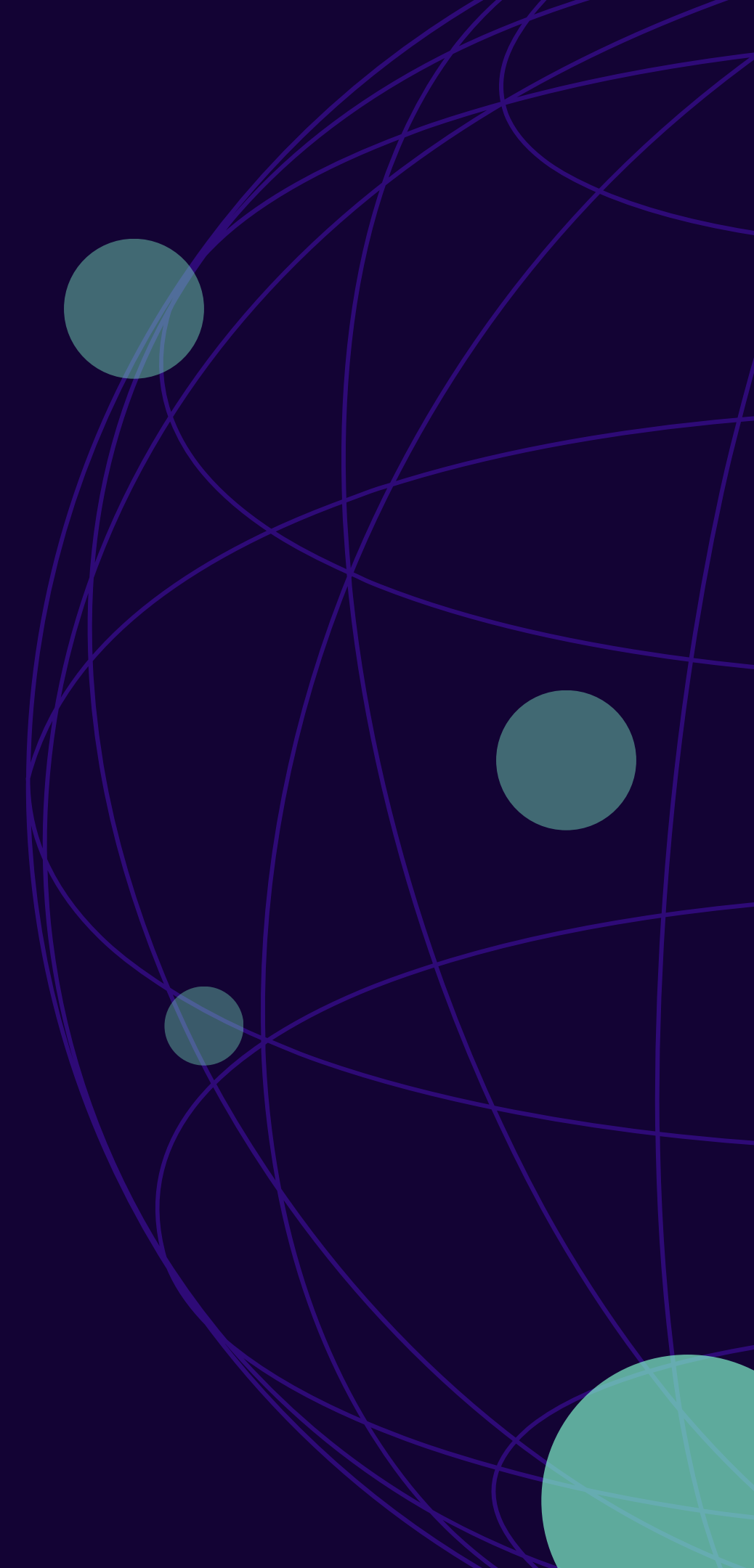
06 La globalización en la política

07 Documentación general

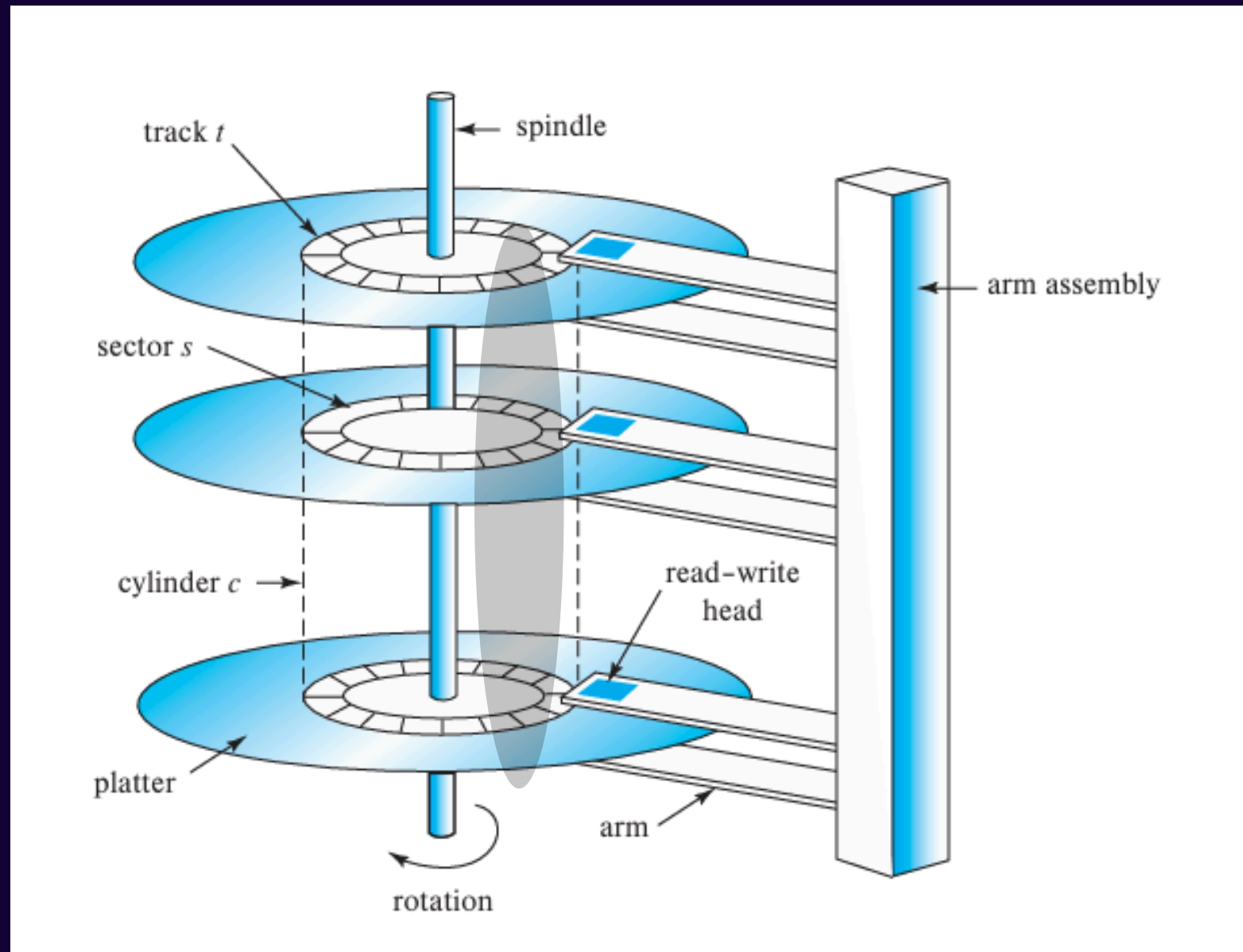
08 Estadísticas principales

09 Los estudios internacionales

10 Conclusiones finales



Estructura del Disco



=== Características del Disco ===

Platos: 4

Superficies x plato: 2

Pistas por superficie: 5

Sectores por pista: 10

Tamaño de sector: 800 bytes

Bloques totales: 50

Tamaño de bloque: 6400 bytes

Capacidad total: 320000 bytes

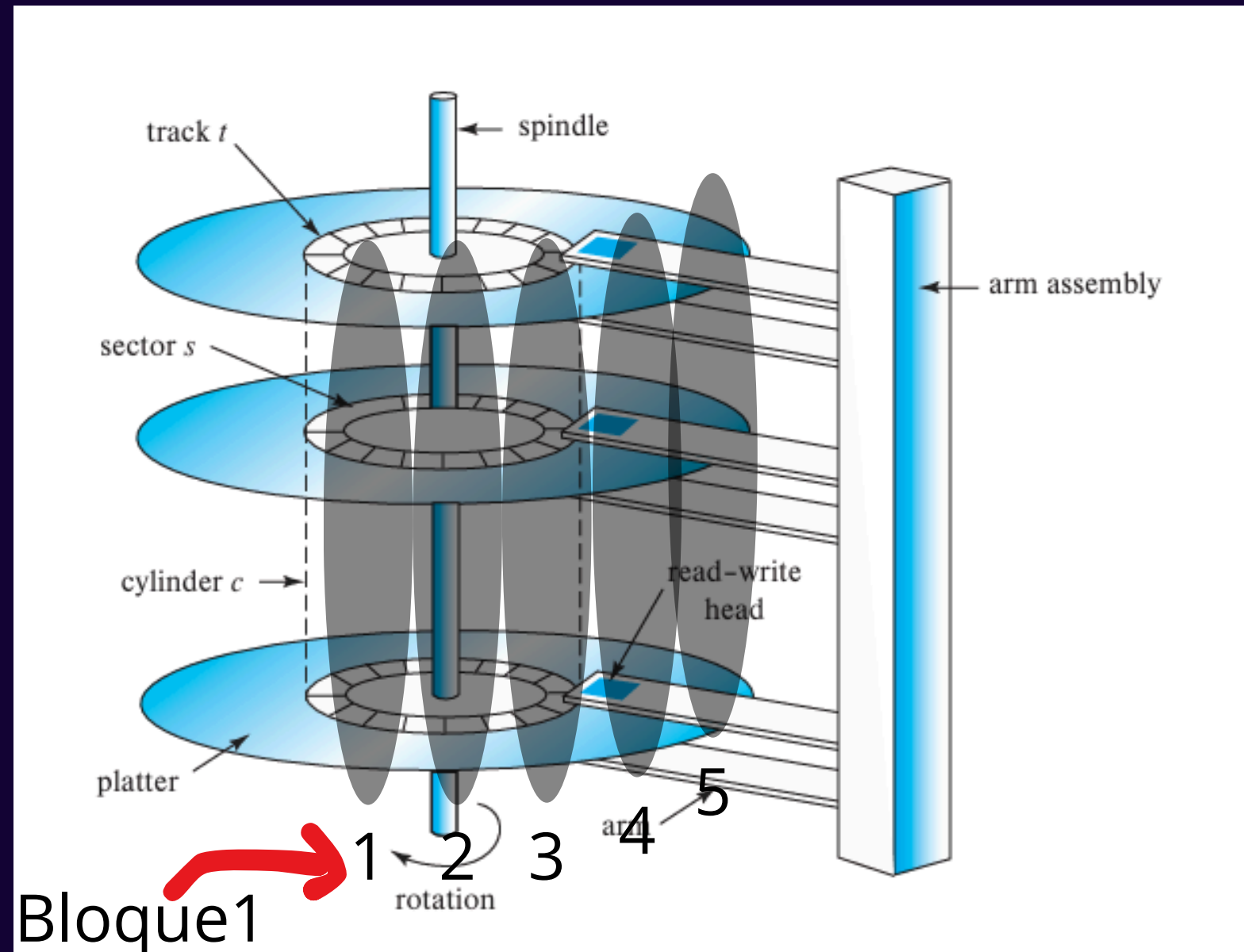
Espacio libre: 320000 bytes

Bloques por pista: 1

Bloques por plato: 10

Silberschatz, A., Korth, H. F., & Sudarshan, S.
Database system concepts.

Estructura del Disco



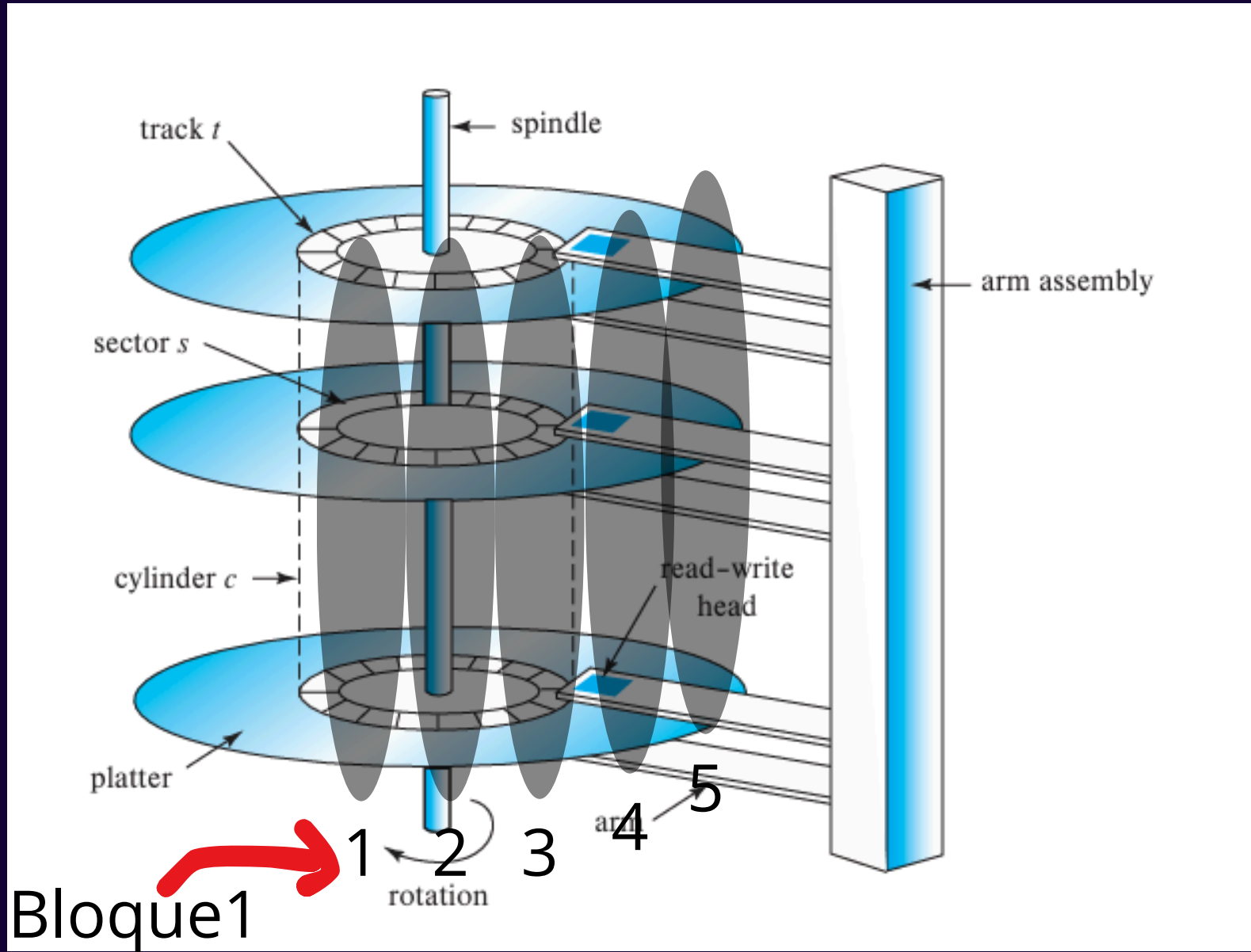
Silberschatz, A., Korth, H. F., & Sudarshan, S.
Database system concepts.

De forma CONTIGUA
los bloques asignados en el disco duro son adyacentes.

```
| 6400#2#BLOQUE#1#6400
```

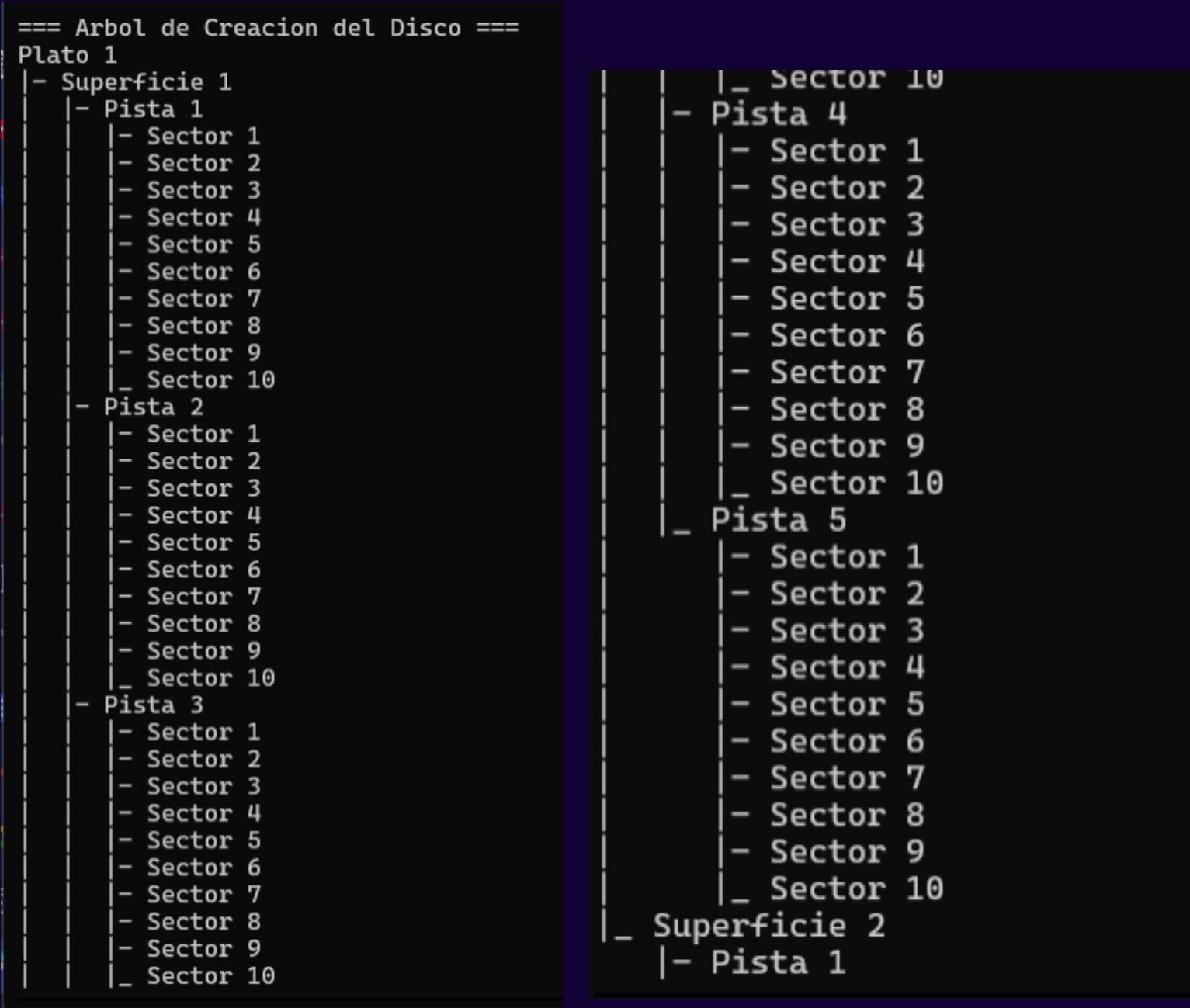
```
#_800#1/1/1/1  
#_800#1/2/1/1  
#_800#2/1/1/1  
#_800#2/2/1/1  
#_800#3/1/1/1  
#_800#3/2/1/1  
#_800#4/1/1/1  
#_800#4/2/1/1  
#_@@@@@@@@@|
```


Estructura del Disco

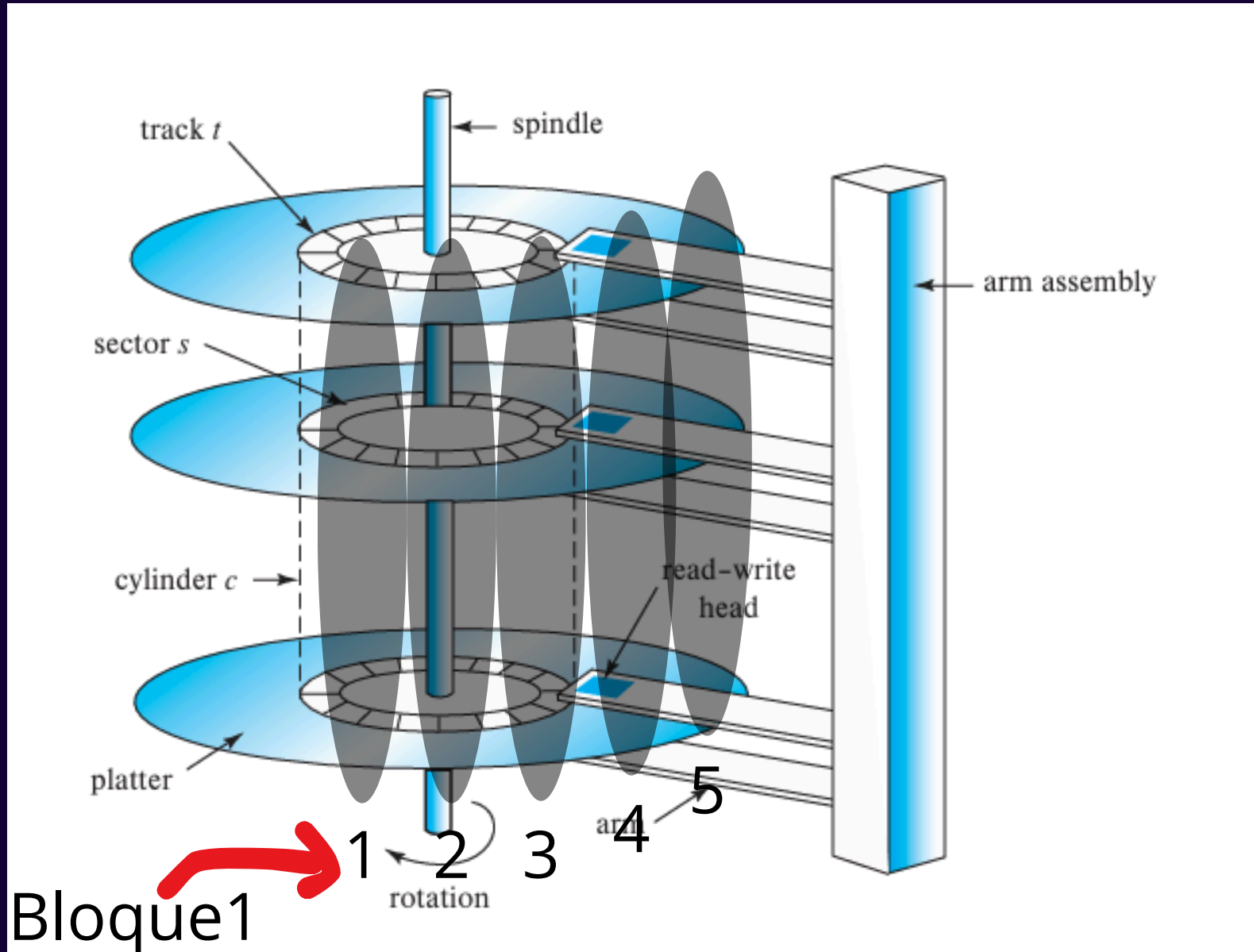


Silberschatz, A., Korth, H. F., & Sudarshan, S.
Database system concepts.

De forma CONTIGUA
los bloques asignados en el disco duro son adyacentes.

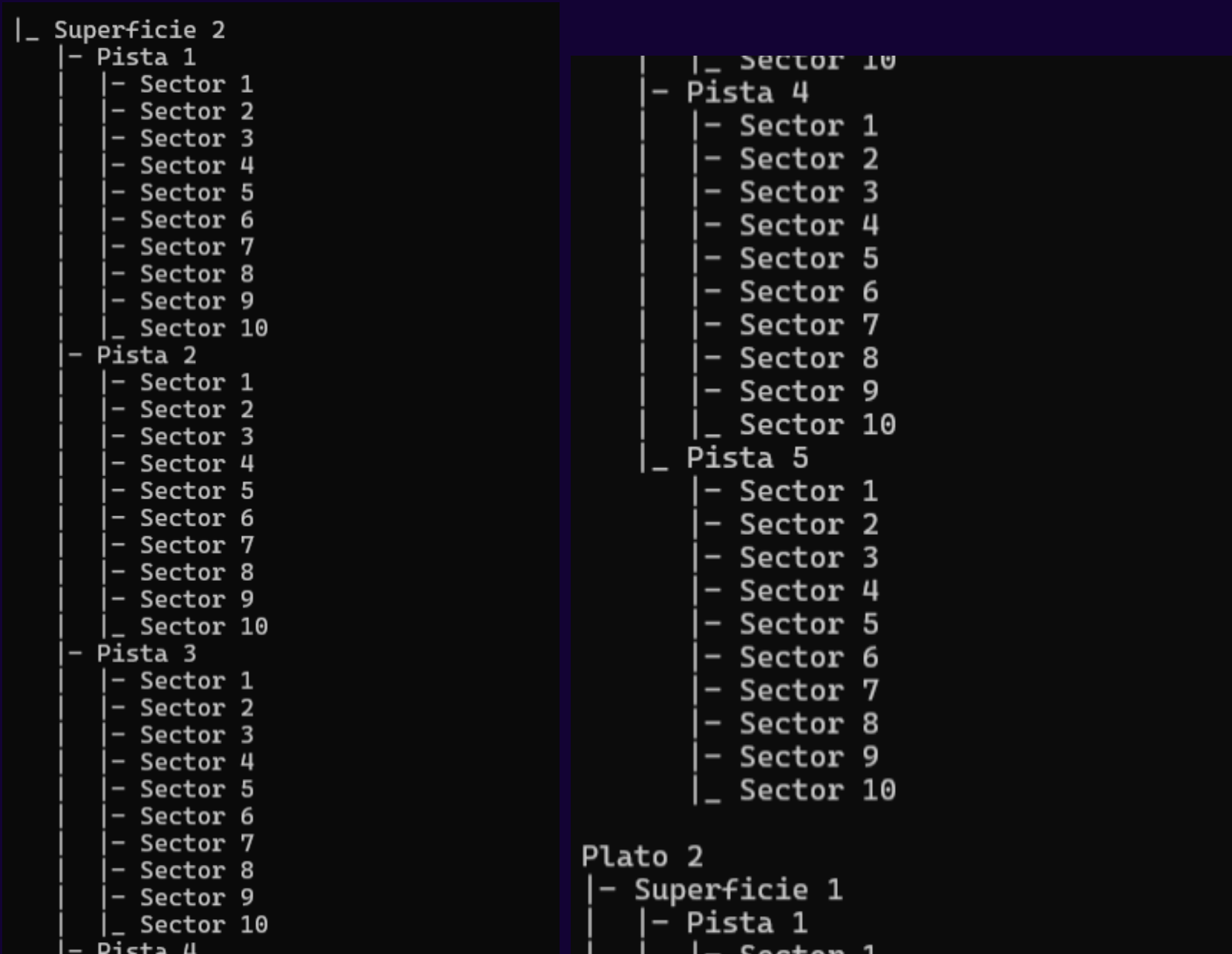


Estructura del Disco

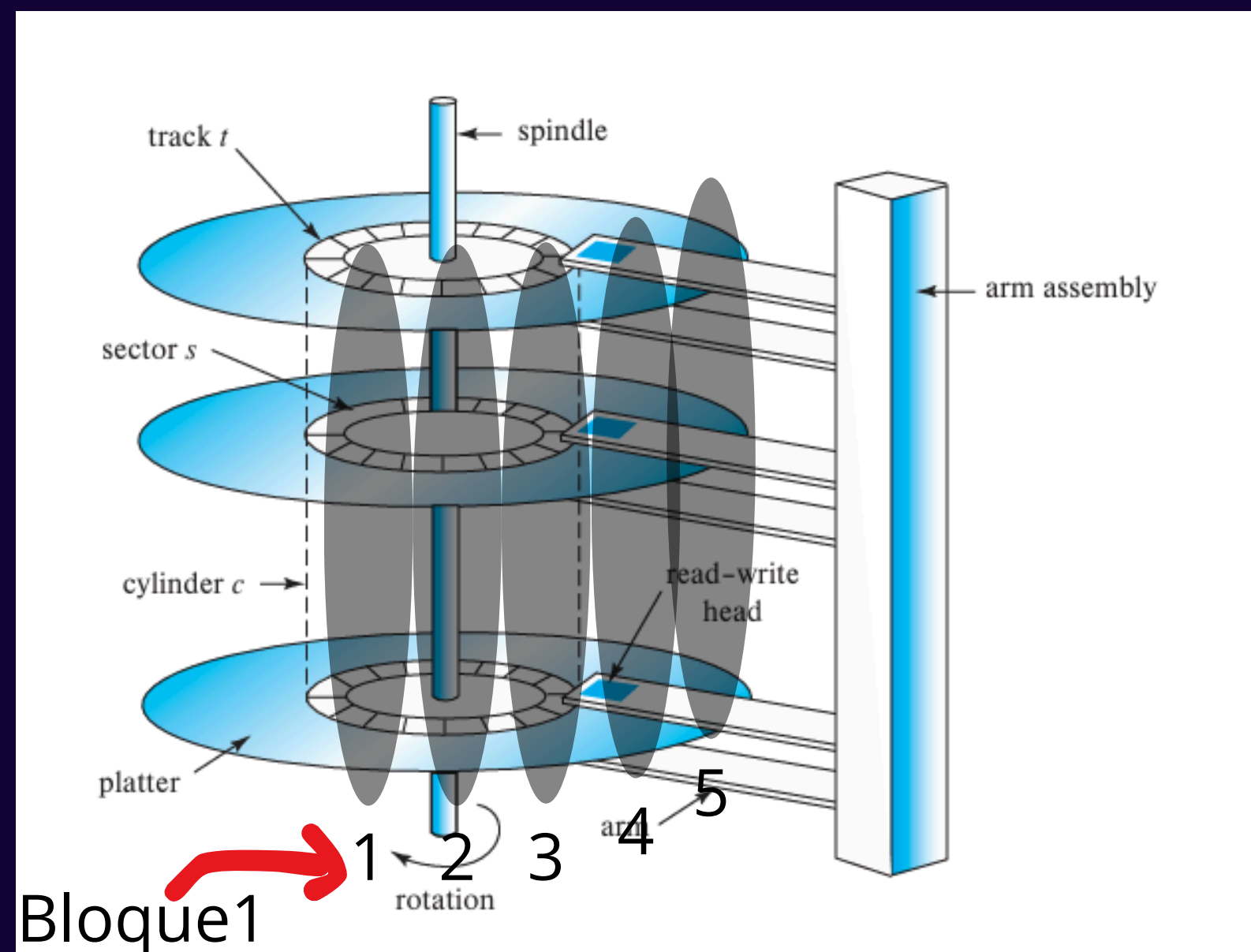


Silberschatz, A., Korth, H. F., & Sudarshan, S.
Database system concepts.

De forma CONTIGUA
los bloques asignados en el disco duro son adyacentes.



Estructura del Disco



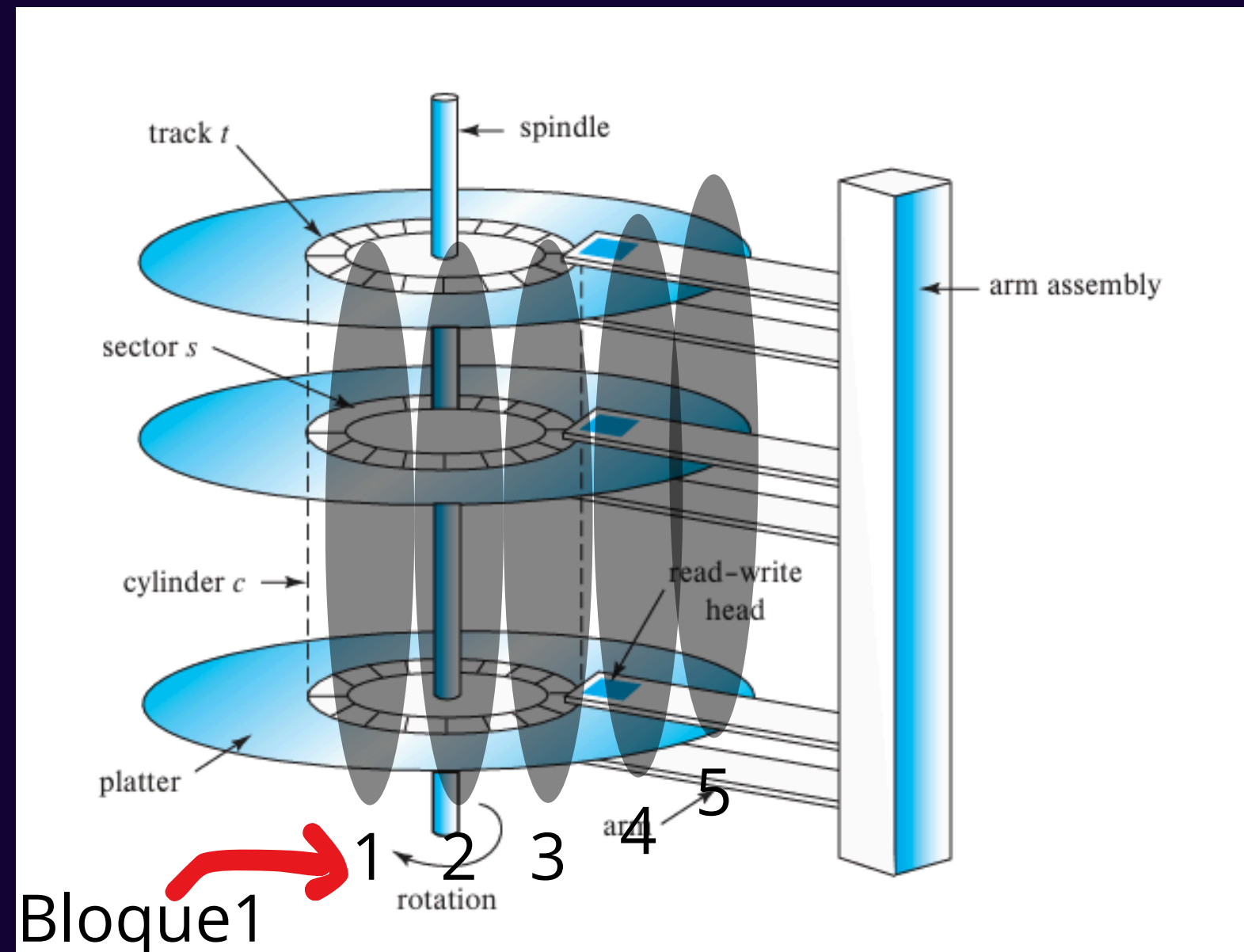
Silberschatz, A., Korth, H. F., & Sudarshan, S.
Database system concepts.

De forma CONTIGUA
los bloques asignados en el disco duro son adyacentes.

Plato 4	
- Superficie 1	
- Pista 1	- Sector 1
	- Sector 2
	- Sector 3
	- Sector 4
	- Sector 5
	- Sector 6
	- Sector 7
	- Sector 8
	- Sector 9
	- Sector 10
- Pista 2	- Sector 1
	- Sector 2
	- Sector 3
	- Sector 4
	- Sector 5
	- Sector 6
	- Sector 7
	- Sector 8
	- Sector 9
	- Sector 10
- Pista 3	- Sector 1
	- Sector 2
	- Sector 3
	- Sector 4
	- Sector 5
	- Sector 6
	- Sector 7
	- Sector 8

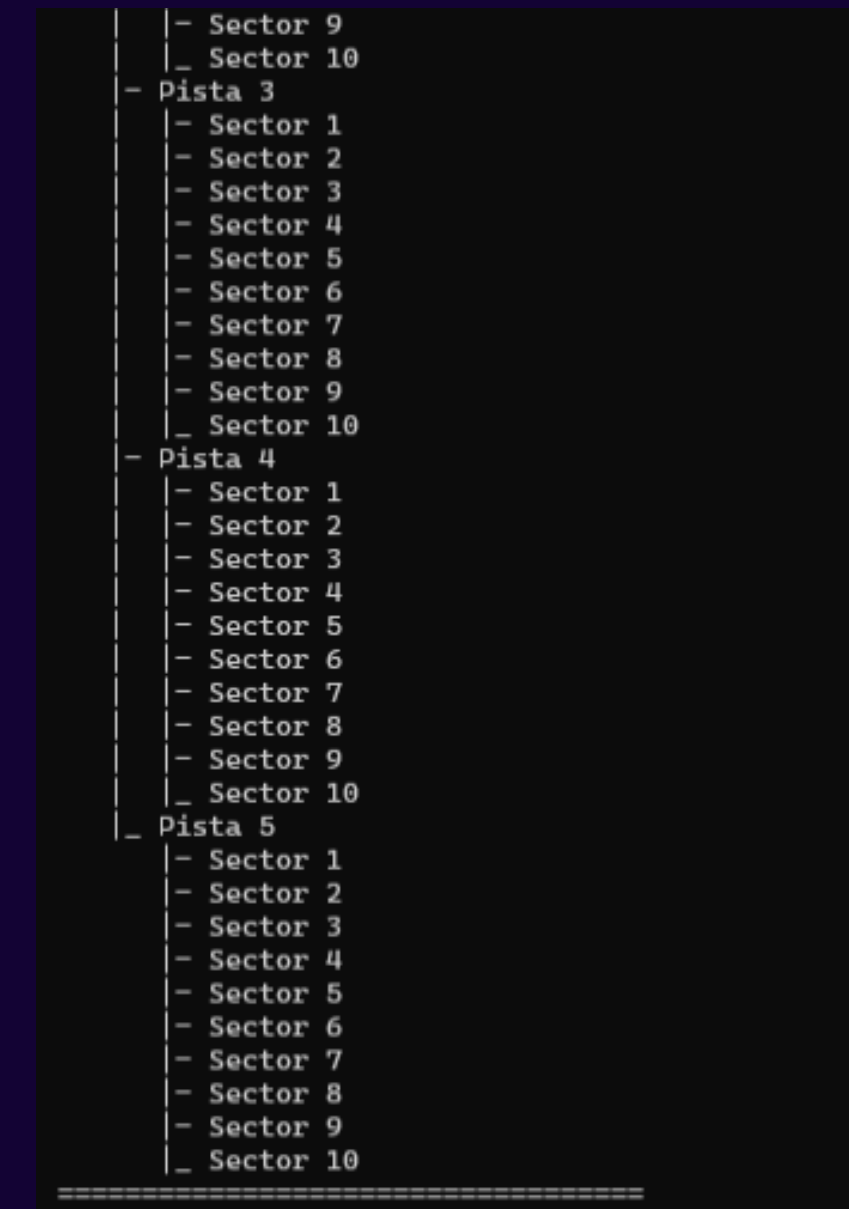
- Sector 9	
- Sector 10	
- Pista 4	- Sector 1
	- Sector 2
	- Sector 3
	- Sector 4
	- Sector 5
	- Sector 6
	- Sector 7
	- Sector 8
	- Sector 9
	- Sector 10
- Pista 5	- Sector 1
	- Sector 2
	- Sector 3
	- Sector 4
	- Sector 5
	- Sector 6
	- Sector 7
	- Sector 8
	- Sector 9
	- Sector 10
- Superficie 2	
- Pista 1	- Sector 1
	- Sector 2
	- Sector 3
	- Sector 4
	- Sector 5
	- Sector 6
	- Sector 7
	- Sector 8
	- Sector 9
	- Sector 10
- Pista 2	- Sector 1
	- Sector 2
	- Sector 3
	- Sector 4
	- Sector 5
	- Sector 6
	- Sector 7
	- Sector 8
	- Sector 9
	- Sector 10
- Pista 3	- Sector 1

Estructura del Disco



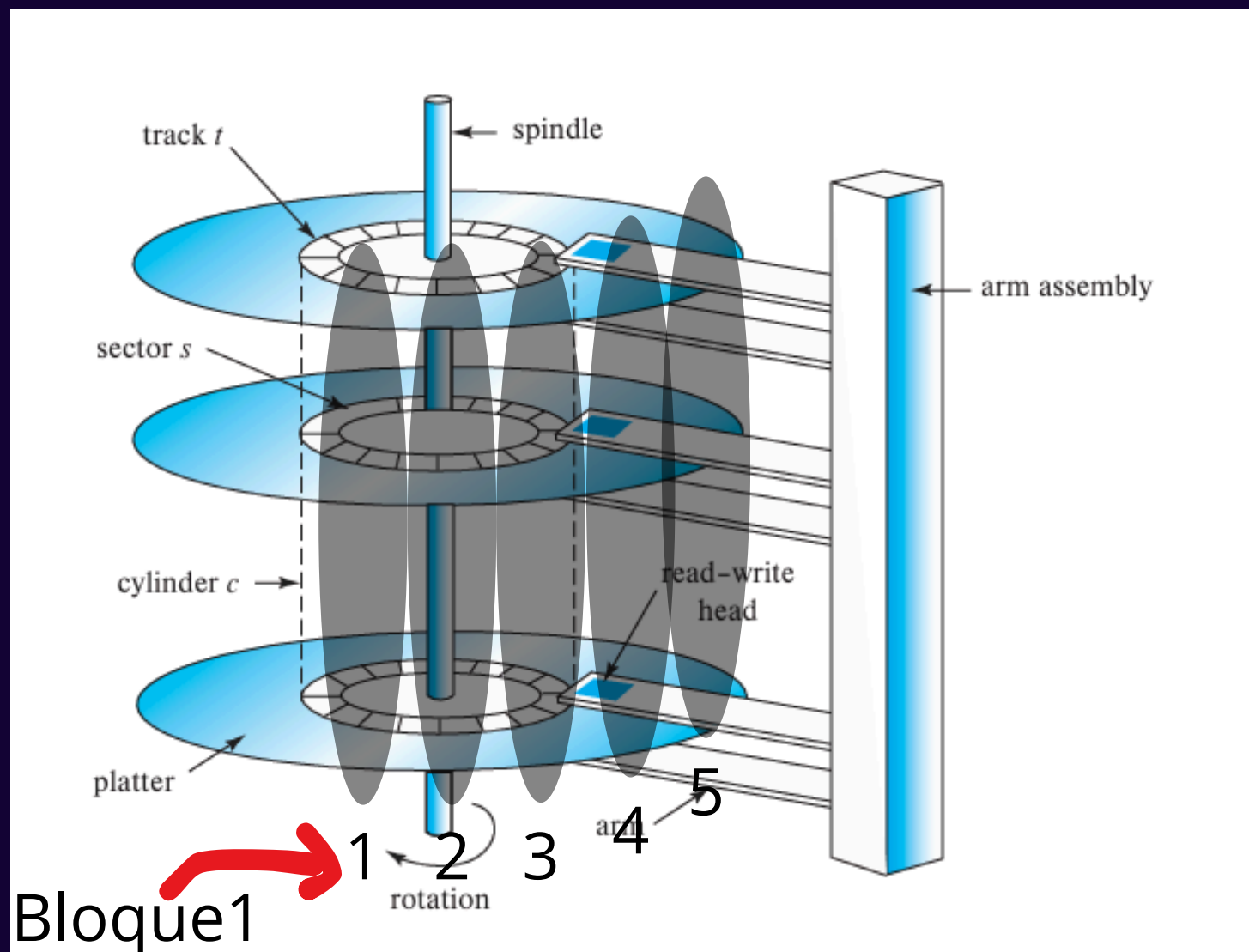
Silberschatz, A., Korth, H. F., & Sudarshan, S.
Database system concepts.

De forma CONTIGUA
los bloques asignados en el disco duro son adyacentes.



Estructura del Disco

Luego de adicionar todo el .CSV (Housing)



Silberschatz, A., Korth, H. F., & Sudarshan, S.
Database system concepts.

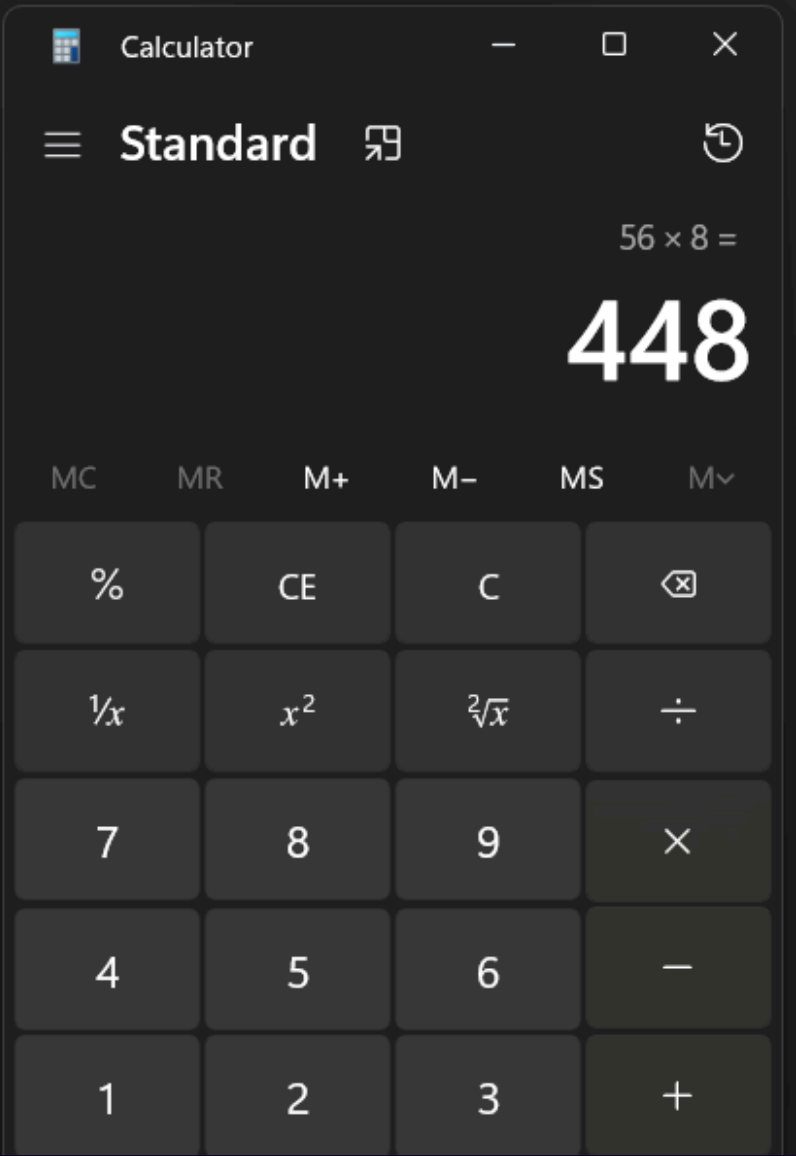
Espacio Libre Bloque

Espacio Libre Sector

| 448#2#BLOQUE#3#6400

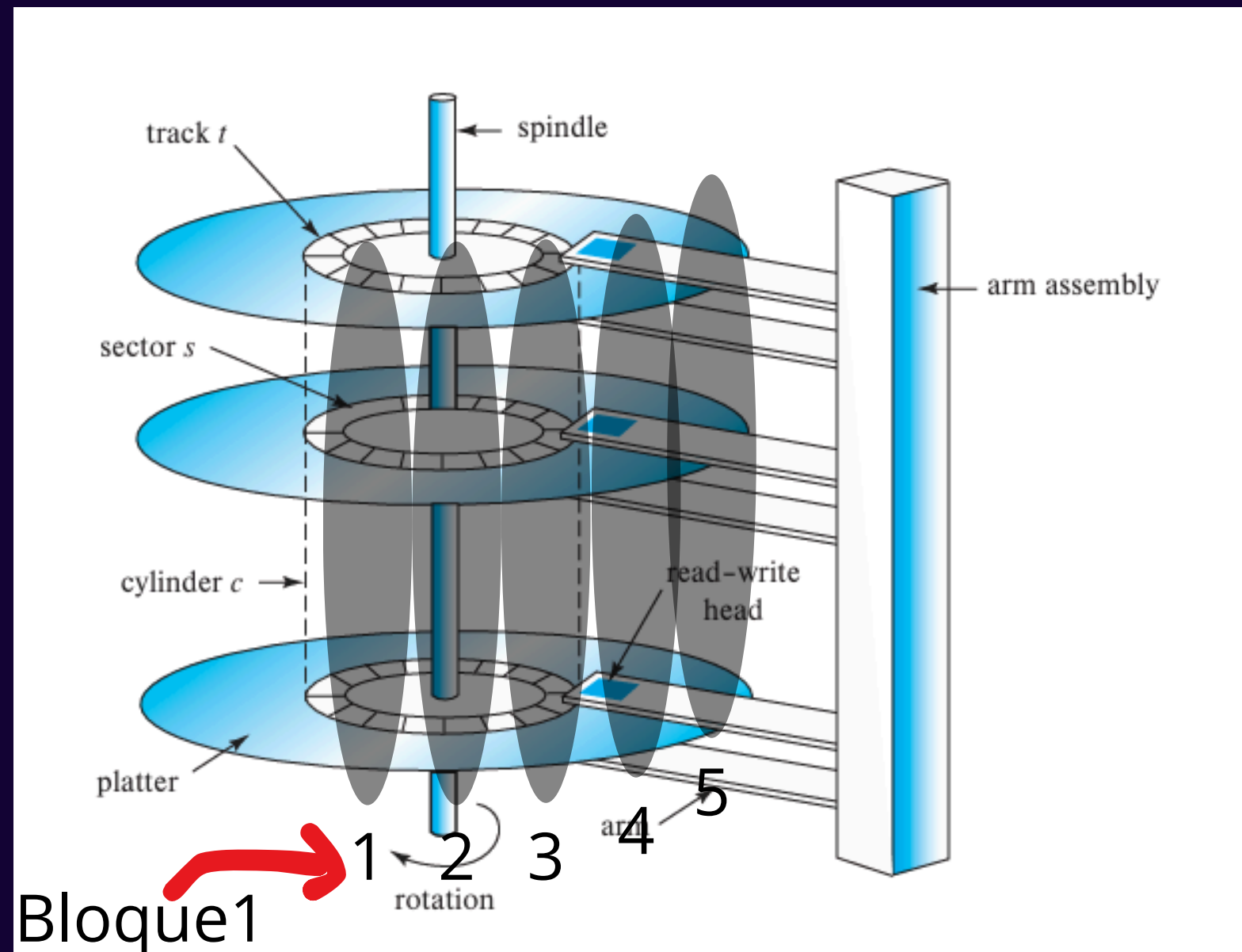
#_56#1/1/1/3
#_56#1/2/1/3
#_56#2/1/1/3
#_56#2/2/1/3
#_56#3/1/1/3
#_56#3/2/1/3
#_56#4/1/1/3
#_56#4/2/1/3
#_@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@|

Ruta del sector
(PATH)



Uso de la teoria de Longitud Fija para evitar desplazamientos

Estructura del Disco



Silberschatz, A., Korth, H. F., & Sudarshan, S.
Database system concepts.

De forma CONTIGUA
los bloques asignados en el disco duro son adyacentes.

```
| 6400#2#BLOQUE#50#6400
```

```
#_800#1/1/5/10
```

```
#_800#1/2/5/10
```

```
#_800#2/1/5/10
```

```
#_800#2/2/5/10
```

```
#_800#3/1/5/10
```

```
#_800#3/2/5/10
```

```
#_800#4/1/5/10
```

```
#_800#4/2/5/10#_||
```

Sin @ ya que este formato Bloque llega a cubrir la longitud
Maxima

Estructura del Disco

```
| 6400#2#BLOQUE#50#6400

#_800#1/1/5/10
#_800#1/2/5/10
#_800#2/1/5/10
#_800#2/2/5/10
#_800#3/1/5/10
#_800#3/2/5/10
#_800#4/1/5/10
#_800#4/2/5/10#_||
```

Sin @ ya que este formato Bloque llega a cubrir la longitud Maxima

De forma CONTIGUA
los bloques asignados en el disco duro son adyacentes.

```
| 6400#2#BLOQUE#50#6400#_800#1/1/5/10#_800#1/2/5/10#_800#1/3/2/5/10#_800#4/1/5/10#_800#4/2/5/10#_||
```

Ln 7, Col 138 137 of 6,856 characters

```
| 448#2#BLOQUE#1#6400#_56#1/1/1/1#_56#1/2/1/1#_56#2/1/1/1/1#_56#4/2/1/1#_@@@@@@@@@@@@@@@@@@@@@|
```

Ln 22, Col 138 137 of 6,891 characters



Primera Parte



Segunda Parte

Registros Longitud Fija y Variable

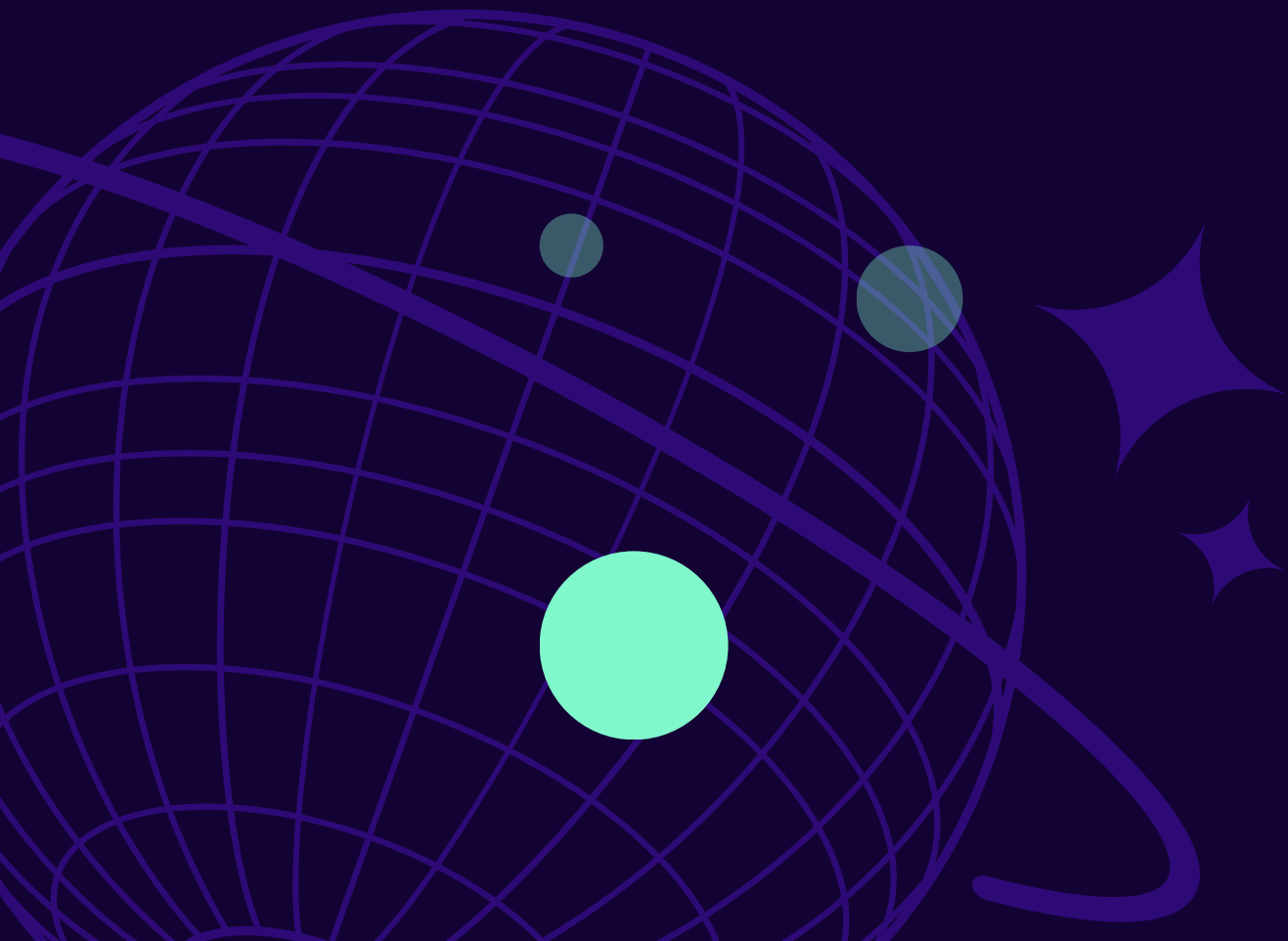
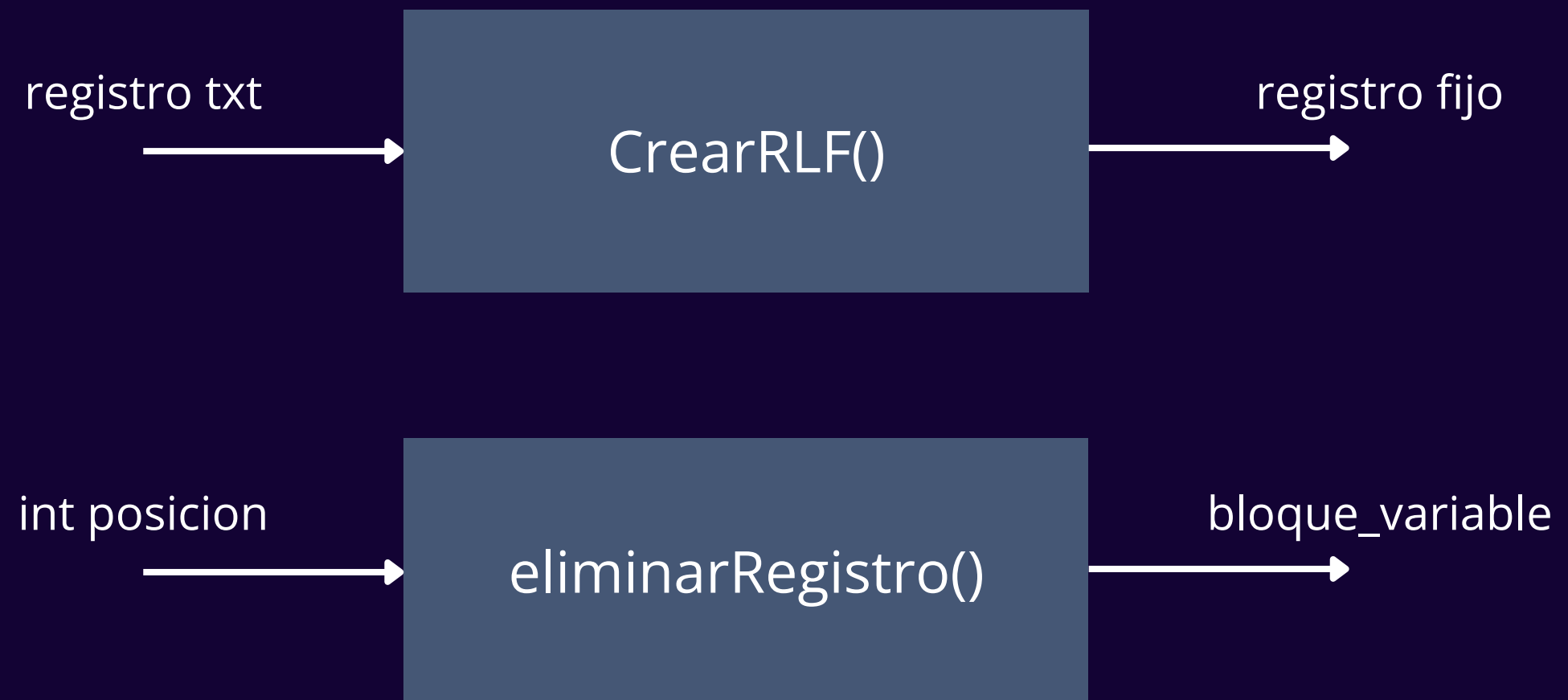
Cabeceras explicar nuestro metodo
(ejm: slotted-page)



Formato del bloque



Funciones




```
#yes#yes#no@#no@#yes#1#no@#furnished@@@@@#| 7140000@#6000
#yes#no@#no@#no@#yes#1#no@#furnished@@@@@#| 7035000@#6360
#yes#no@#yes#no@#yes#1#no@#furnished@@@@@#| 6930000@#1320
#yes#no@#no@#no@#yes#1#no@#furnished@@@@@#| 6790000@#1200
#yes#no@#no@#no@#yes#0#no@#unfurnished@@@#| 6720000@#5020
#yes#no@#yes#yes#no@#1#no@#furnished@@@@@#| 6650000@#4260
#yes#no@#no@#no@#yes#0#yes#furnished@@@@@#| 6650000@#5700
#yes#no@#no@#yes#no@#1#yes#semi-furnished#| 6615000@#4000
#yes#no@#no@#no@#yes#0#no@#semi-furnished#| 6510000@#3760
#yes#no@#yes#no@#no@#0#yes#unfurnished@@@#| 6475000@#3960
#yes#no@#no@#no@#no@#2#no@#furnished@@@@@#| 6440000@#5000
#yes#yes#no@#no@#yes#0#no@#furnished@@@@@#| 6300000@#7200
#yes#no@#no@#no@#yes#2#no@#semi-furnished#|
```


Registros Longitud Fija


```
#no@#no@#no@#yes#no@#0#no@#unfurnished@@@#| 2233000@#5300@
#yes#no@#no@#no@#no@#0#no@#unfurnished@@@#| 2100000@#3000@
#no@#no@#no@#no@#no@#0#no@#unfurnished@@@#| 1890000@#1700@
#no@#no@#no@#no@#no@#1#no@#unfurnished@@@#| 1820000@#3000@
#yes#no@#no@#no@#no@#0#no@#unfurnished@@@#| 1750000@#2910@
```


Ultimo Sector Volcado.txt

Sector6.txt

File Edit View

1855000@#2990@#2#1#1#no@#no@#no@#no@#no@#1#no@#unfurnished@@@#|1820000@#3000@#2#1#1#yes#no@#yes#no@#no@#2#no@#unfurnished@@@#|
1767150@#2400@#3#1#1#no@#no@#no@#no@#no@#0#no@#semi-furnished#|1750000@#3620@#2#1#1#yes#no@#no@#no@#no@#0#no@#unfurnished@@@#|
1750000@#2910@#3#1#1#no@#no@#no@#no@#no@#0#no@#furnished@@@@#|1750000@#3850@#3#1#2#yes#no@#no@#no@#no@#0#no@#unfurnished@@@#|

D:\DBPlusPlus\DBPlusPlus\DISCO\Plato3\S2\Pista1

C:\Windows\Minidump

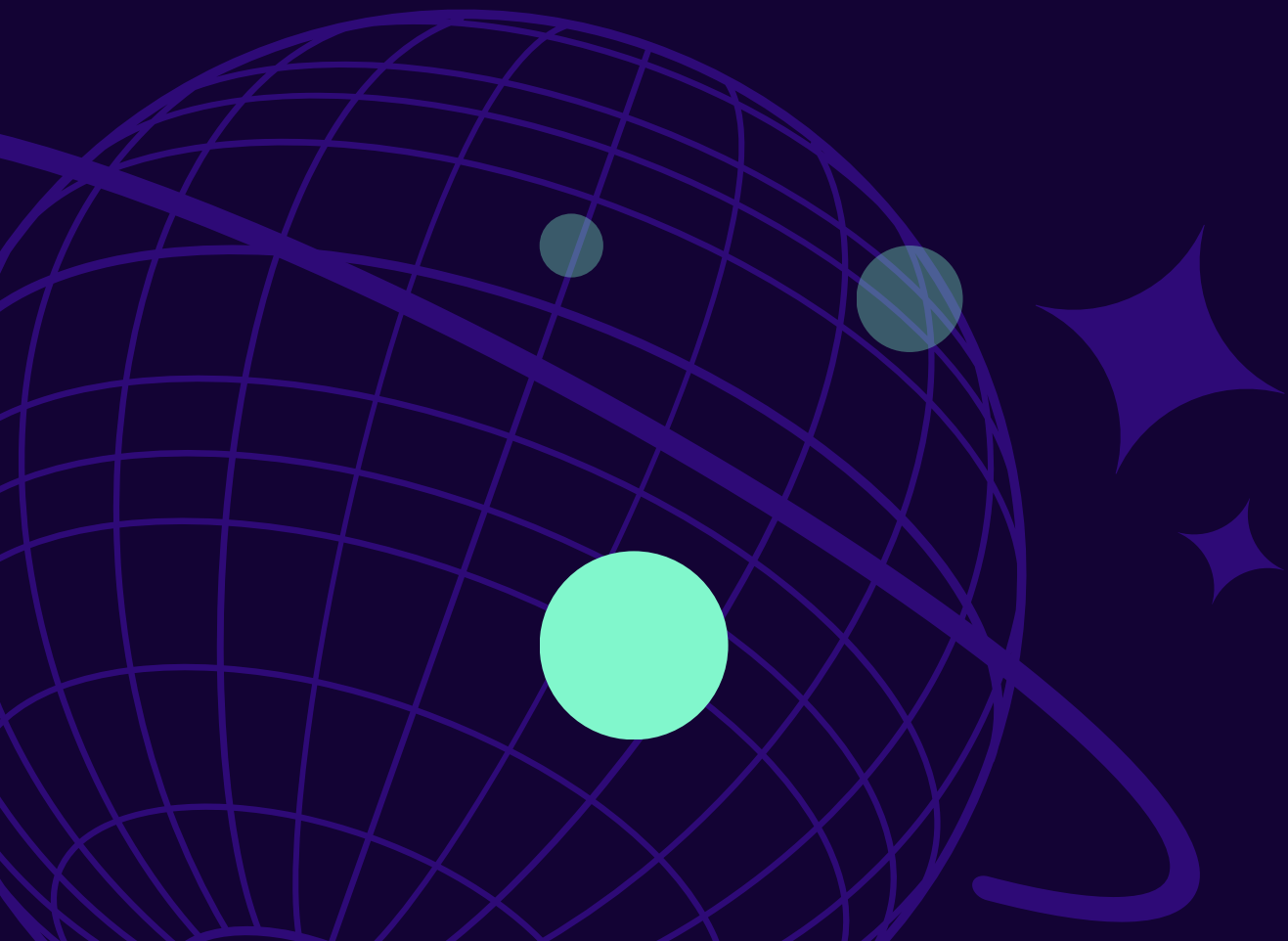
Nombre	Fecha de modificación	Tipo	Tamaño
Sector1.txt	05/06/2025 03:33 a. m.	Text Document	1 KB
Sector2.txt	05/06/2025 03:33 a. m.	Text Document	1 KB
Sector3.txt	05/06/2025 03:33 a. m.	Text Document	1 KB
Sector4.txt	05/06/2025 03:33 a. m.	Text Document	1 KB
Sector5.txt	05/06/2025 03:33 a. m.	Text Document	1 KB
Sector6.txt	05/06/2025 03:33 a. m.	Text Document	1 KB
Sector7.txt	05/06/2025 03:32 a. m.	Text Document	0 KB
Sector8.txt	05/06/2025 03:32 a. m.	Text Document	0 KB
Sector9.txt	05/06/2025 03:32 a. m.	Text Document	0 KB
Sector10.txt	05/06/2025 03:32 a. m.	Text Document	0 KB

| 2370#2#BLOQUE#6#6400

#_56#1/1/1/6
#_56#1/2/1/6
#_56#2/1/1/6
#_56#2/2/1/6
#_56#3/1/1/6
#_490#3/2/1/6
#_800#4/1/1/6
#_800#4/2/1/6
#_@@@@@@@@@@@@@@@@@|

Como Estructuramos los Bloques/Paginas (PATHs)

Bloques relacionados con sectores (path)



Como Estructuramos los Bloques/Paginas (PATHs)

Bloques relacionados con secto

```
static bool crearRLF(const char* registroTxt, const char* relacion, char* outBuffer, int* outLen) {
    int numFields = 0;
    int maxLen[MAX_FIELDS] = { 0 };
    printf("DEBUG: Llamando a crearRLF para relación '%s' con registroTxt: '%s'\n", relacion, registroTxt);

    if (!obtenerLongitudesPorCampo(relacion, &numFields, maxLen)) {
        printf("DEBUG: obtenerLongitudesPorCampo falló para '%s'\n", relacion);
        return false;
    }
    printf("DEBUG: Longitudes por campo para '%s': ", relacion);
    for (int i = 0; i < numFields; ++i) {
        printf("%d ", maxLen[i]);
    }
    printf("\n");

    char copy[MAX_BUF];
    strncpy(copy, registroTxt, MAX_BUF - 1);
    copy[MAX_BUF - 1] = '\0';

    char* tok = strtok(copy, "#");
    int ofs = 0;

    for (int i = 0; i < numFields; i++) {
        if (!tok) {
            printf("DEBUG: Faltan campos en registroTxt para el campo %d\n", i);
            return false;
        }

        int lenVal = (int)strlen(tok);
        int fieldLen = maxLen[i];
        int toCopy = (lenVal < fieldLen) ? lenVal : fieldLen;

        // Copiamos el contenido real (o truncado) del campo
        memcpy(outBuffer + ofs, tok, toCopy);

        // Si el valor es más corto, rellenamos con '@'
        if (lenVal < fieldLen) {
            for (int k = toCopy; k < fieldLen; k++) {
                outBuffer[ofs + k] = '@';
            }
        }

        // Debug: mostramos cómo queda el campo antes del '#'
        printf("DEBUG: Campo %d (fijo): '", i);
        for (int k = 0; k < fieldLen; ++k) {
            if (k < toCopy) putchar(tok[k]);
            else putchar('@');
        }
        printf("\n");

        ofs += fieldLen;

        // Insertamos el separador '#' justo después de cada campo
        outBuffer[ofs] = '#';
        ofs += 1;

        // Debug: mostramos el '#' insertado
        printf("DEBUG: Añadido separador '#'\n");

        // Continuamos al siguiente token
        tok = strtok(NULL, "#");
    }

    *outLen = ofs;

    // Mostrar el registro fijo completo (con '#' tras cada campo)
    printf("DEBUG: Registro fijo generado (longitud %d): [", *outLen);
    for (int i = 0; i < *outLen; ++i) {
        putchar(outBuffer[i]);
    }
    printf("]\n");

    return true;
}
```

```
// Copiamos el contenido real (o truncado) del campo
memcpy(outBuffer + ofs, tok, toCopy);

// Si el valor es más corto, rellenamos con '@'
if (lenVal < fieldLen) {
    for (int k = toCopy; k < fieldLen; k++) {
        outBuffer[ofs + k] = '@';
    }
}

// Debug: mostramos cómo queda el campo antes del '#'
printf("DEBUG: Campo %d (fijo): '", i);
for (int k = 0; k < fieldLen; ++k) {
    if (k < toCopy) putchar(tok[k]);
    else putchar('@');
}
printf("\n");

ofs += fieldLen;

// Insertamos el separador '#' justo después de cada campo
outBuffer[ofs] = '#';
ofs += 1;

// Debug: mostramos el '#' insertado
printf("DEBUG: Añadido separador '#'\n");

// Continuamos al siguiente token
tok = strtok(NULL, "#");
}

*outLen = ofs;

// Mostrar el registro fijo completo (con '#' tras cada campo)
printf("DEBUG: Registro fijo generado (longitud %d): [", *outLen);
for (int i = 0; i < *outLen; ++i) {
    putchar(outBuffer[i]);
}
printf("]\n");

return true;
}
```

Como Estructuramos los Bloques/Paginas (PATHs)

Bloques relacionados con secto

```
static bool crearRLF(const char* registroTxt, const char* relacion, char* outBuffer, int* outLen) {
    int numFields = 0;
    int maxLen[MAX_FIELDS] = { 0 };
    printf("DEBUG: Llamando a crearRLF para relación '%s' con registroTxt: '%s'\n", relacion, registroTxt);

    if (!obtenerLongitudesPorCampo(relacion, &numFields, maxLen)) {
        printf("DEBUG: obtenerLongitudesPorCampo falló para '%s'\n", relacion);
        return false;
    }
    printf("DEBUG: Longitudes por campo para '%s': ", relacion);
    for (int i = 0; i < numFields; ++i) {
        printf("%d ", maxLen[i]);
    }
    printf("\n");

    char copy[MAX_BUF];
    strncpy(copy, registroTxt, MAX_BUF - 1);
    copy[MAX_BUF - 1] = '\0';

    char* tok = strtok(copy, "#");
    int ofs = 0;

    for (int i = 0; i < numFields; i++) {
        if (!tok) {
            printf("DEBUG: Faltan campos en registroTxt para el campo %d\n", i);
            return false;
        }

        int lenVal = (int)strlen(tok);
        int fieldLen = maxLen[i];
        int toCopy = (lenVal < fieldLen) ? lenVal : fieldLen;

        // Copiamos el contenido real (o truncado) del campo
        memcpy(outBuffer + ofs, tok, toCopy);

        // Si el valor es más corto, rellenamos con '@'
        if (lenVal < fieldLen) {
            for (int k = toCopy; k < fieldLen; k++) {
                outBuffer[ofs + k] = '@';
            }
        }

        // Debug: mostramos cómo queda el campo antes del '#'
        printf("DEBUG: Campo %d (fijo): '", i);
        for (int k = 0; k < fieldLen; ++k) {
            if (k < toCopy) putchar(tok[k]);
            else putchar('@');
        }
        printf("\n");

        ofs += fieldLen;

        // Insertamos el separador '#' justo después de cada campo
        outBuffer[ofs] = '#';
        ofs += 1;

        // Debug: mostramos el '#' insertado
        printf("DEBUG: Añadido separador '#'\n");

        // Continuamos al siguiente token
        tok = strtok(NULL, "#");
    }

    *outLen = ofs;

    // Mostrar el registro fijo completo (con '#' tras cada campo)
    printf("DEBUG: Registro fijo generado (longitud %d): [", *outLen);
    for (int i = 0; i < *outLen; ++i) {
        putchar(outBuffer[i]);
    }
    printf("]\n");

    return true;
}
```

```
// Copiamos el contenido real (o truncado) del campo
memcpy(outBuffer + ofs, tok, toCopy);

// Si el valor es más corto, rellenamos con '@'
if (lenVal < fieldLen) {
    for (int k = toCopy; k < fieldLen; k++) {
        outBuffer[ofs + k] = '@';
    }
}

// Debug: mostramos cómo queda el campo antes del '#'
printf("DEBUG: Campo %d (fijo): '", i);
for (int k = 0; k < fieldLen; ++k) {
    if (k < toCopy) putchar(tok[k]);
    else putchar('@');
}
printf("\n");

ofs += fieldLen;

// Insertamos el separador '#' justo después de cada campo
outBuffer[ofs] = '#';
ofs += 1;

// Debug: mostramos el '#' insertado
printf("DEBUG: Añadido separador '#'\n");

// Continuamos al siguiente token
tok = strtok(NULL, "#");
}

*outLen = ofs;

// Mostrar el registro fijo completo (con '#' tras cada campo)
printf("DEBUG: Registro fijo generado (longitud %d): [", *outLen);
for (int i = 0; i < *outLen; ++i) {
    putchar(outBuffer[i]);
}
printf("]\n");

return true;
}
```

Registros Longitud Variable

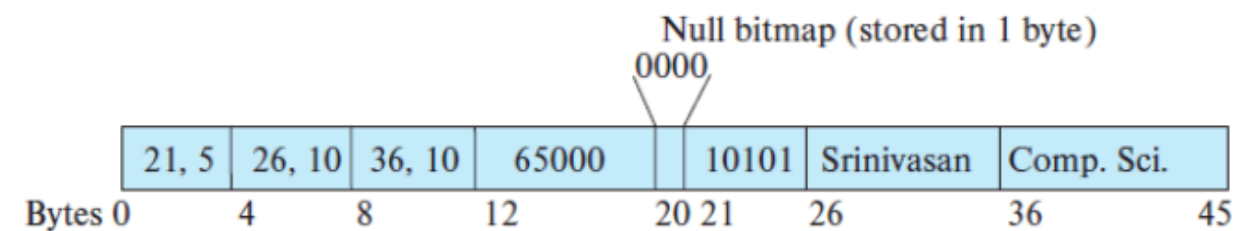
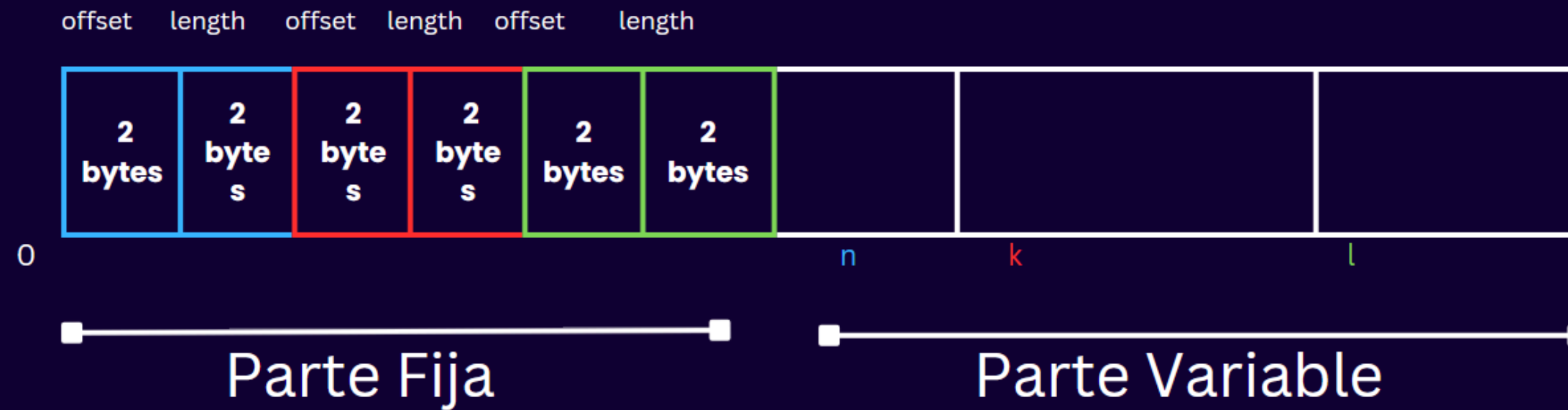


Figure 13.5 Representation of a variable-length record of the *instructor* relation.

Formato del bloque

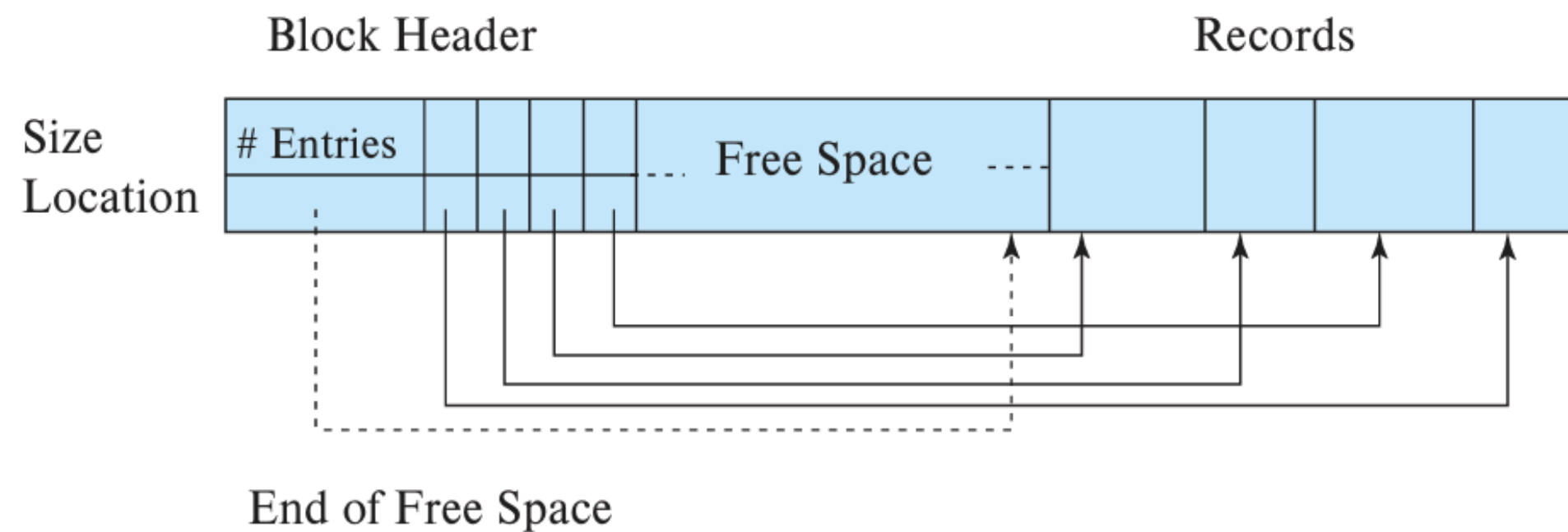
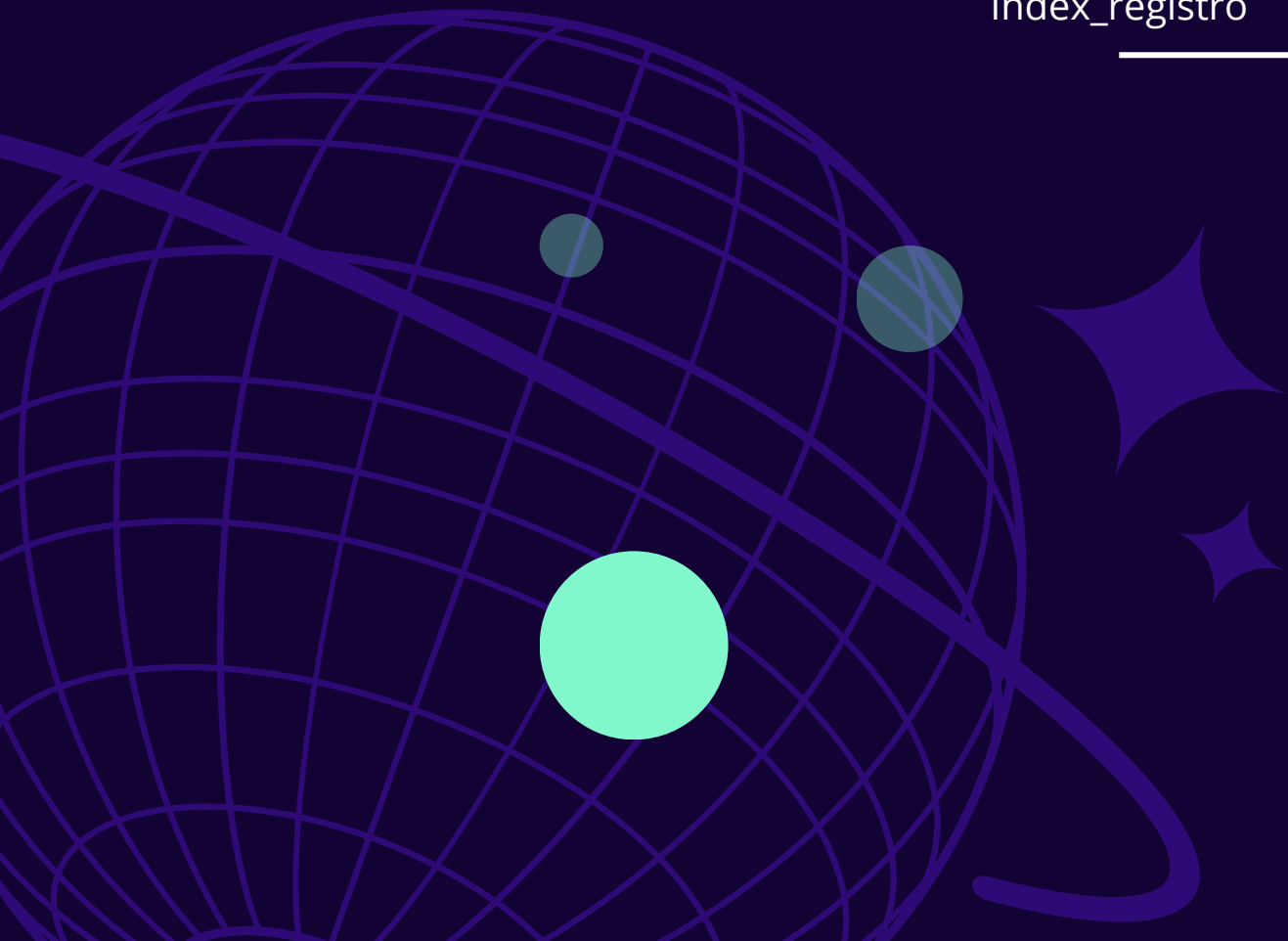
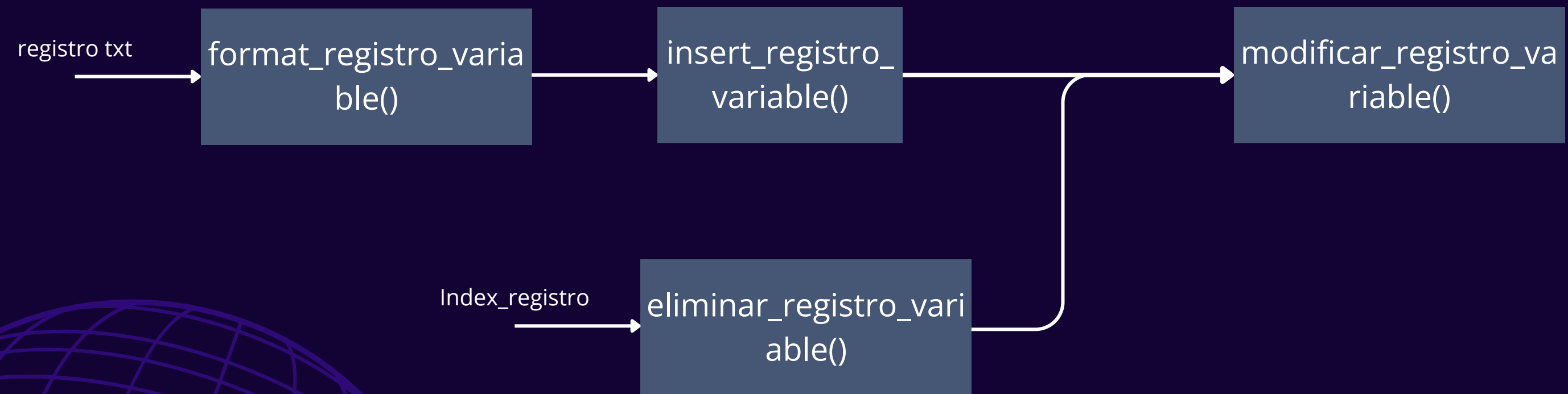


Figure 13.6 Slotted-page structure.

Silberschatz, A., Korth, H. F., & Sudarshan, S. Database system concepts.

Funciones



Funciones: format_registro_variable()

```
char* format_registro_variable(const char registro[], const char relacion[]) {  
    // 1) Obtener número de atributos según "relacion"  
    int field_number = get_n_attributes(relacion);  
  
    // 2) Separar los campos de texto (división por '#')  
    std::vector<std::string> fields;  
    const char* start = registro;  
    for (;;) { ...  
        // Si la cantidad real de campos difiere de field_number, lo ajustamos  
        if ((int)fields.size() != field_number) { ...  
  
        // 3) Calcular el tamaño de cada campo y su offset relativo sin contar el "header".  
        std::vector<int> sizes(field_number);  
        std::vector<int> offsetWithoutHeader(field_number);  
        for (int i = 0; i < field_number; ++i) { ...  
  
        // 4) Iterar para determinar cuánto ocupa en texto (headerLen) la parte "offset|siz  
        int headerLen = 0;  
        int prevHeaderLen = -1;  
        std::vector<int> offsets(field_number, 0);  
  
        while (headerLen != prevHeaderLen) { ...  
    }
```

[DEBUG] actualizar bloques variable finaliza con éxito.

Registro formateado: 65|8|73|4|77|1|78|1|79|1|80|3|83|2|85|2|87|2|89|3|92|1|93|2|95|9|122500008960444yesnononoyes3nofurnished

[DEBUG] registro a modificar: 10 - test por vacío: 522 - old meta len: 08

Funciones: insert_registro_variable()

```
int insert_registro_variable(const char registro_variable[], int& espacio_libre_bloque, char* bloque_variable, Disco& disco) {
    int bloqueSize = disco.getTamBloque();

    // 1) Leer cabecera existente "registro_n|last_pos|offset1|size1|..."
    int registro_n = 0;
    int last_pos = 0;
    int old_meta_len = 0;

    std::vector<int> offsets_old;
    std::vector<int> sizes_old;

    if (bloque_variable[0] >= '0' && bloque_variable[0] <= '9') {
        // a) extraer registro_n
        const char* p = std::strchr(bloque_variable, '|');
        if (!p) {
            registro_n = 0;
            last_pos = bloqueSize - 1;
            old_meta_len = 0;
        } else {
            registro_n = std::stoi(std::string(bloque_variable, p - bloque_variable));
            // b) extraer last_pos
            const char* q = std::strchr(p + 1, '|');
            if (!q) {
                registro_n = 0;
                last_pos = bloqueSize - 1;
                old_meta_len = 0;
            } else {
                last_pos = std::stoi(std::string(p + 1, q - (p + 1)));
                // c) leer pares offset/size
            }
        }
    }
}
```

[illegible]

Funciones: eliminar_registro_variable()

```
bool eliminar_registro_variable(int bloqueN, int r_index, int tamBloque, Disco& disco) {
    // --- 1) Abrir y leer TODO el contenido en fileData (binario) ---
    std::string filename = "DISCO\\BLOQUES\\Bloque" + std::to_string(bloqueN) + ".txt";
    std::ifstream ifs(filename, std::ios::binary);
    if (!ifs.is_open()) {
        std::cerr << "Error: no se pudo abrir " << filename << "\n";
        return false;
    }
    std::ostringstream oss;
    oss << ifs.rdbuf();
    std::string fileData = oss.str();
    ifs.close();

    if (fileData.empty()) {
        // Bloque vacío o formato inválido
        return false;
    }

    // --- 2) Parsear cabecera antigua: "registro_n|last_pos|offset1|size1|offset2|size2|..." ---
    size_t pos1 = fileData.find('|');
    if (pos1 == std::string::npos) return false;
    int registro_n = std::stoi(fileData.substr(0, pos1));

    if (r_index < 1 || r_index > registro_n) {
        // Índice fuera de rango
        return false;
    }

    size_t pos2 = fileData.find('|', pos1 + 1);
    if (pos2 == std::string::npos) return false;
    int last_pos = std::stoi(fileData.substr(pos1 + 1, pos2 - (pos1 + 1)));
```

disco / DBF1USF1US / DBF1USF1US / DISCO / BLOQUES / Bloque1.txt

10|5323|6295|105|6191|104|6080|111|5974|106|5868|106|5756|112|5645|111|5538

9|5428|6295|105|6184|111|6078|106|5972|106|5860|112|5749|111|5642|10

Funciones: modify_registro_variable()

```
bool modify_registro_variable(int NBloque, int r_index, const char relacion[], const char atributo[], const char valor[])
{
    int tamBloque = disco.getTamBloque();
    std::string filename = "DISCO\\BLOQUES\\Bloque" + std::to_string(NBloque) + ".txt";

    // 1) Leer TODO el bloque en modo BINARIO
    std::ifstream ifs(filename, std::ios::binary);
    if (!ifs.is_open()) {
        std::cerr << "Error: no se pudo abrir " << filename << "\n";
        return false;
    }
    std::ostringstream oss;
    oss << ifs.rdbuf(); // copia todos los bytes (incluidos '\0')
    std::string fileData = oss.str();
    ifs.close();

    if (fileData.empty()) {
        std::cerr << "Error: bloque vacío o formato inválido\n";
        return false;
    }

    // 2) Parsear la cabecera para extraer registro_n y los pares (offset, size)
    // Formato: "<registro_n>|<last_pos>|<off1>|<sz1>|<off2>|<sz2>|...|"
    size_t p1 = fileData.find('|');
    if (p1 == std::string::npos) return false;
    int registro_n = std::stoi(fileData.substr(0, p1));
    if (r_index < 1 || r_index > registro_n) {
        std::cerr << "Error: r_index fuera de rango\n";
        return false;
    }
}
```

008960444yesnononoyes3nofurnished65|8|73|4|77|1|78|1|79|1|80|3|83|2|85|2|87|2|89|3|92|1|93|3|96|9|133000007420423yesnononoyes2yes

Conjunto > DBPlusPlus > DBPlusPlus > DISCO > BLOQUES > Bloque1.txt

1 22yesnoyesnono2yessemi-furnished65|8|73|4|77|1|78|1|79|1|80|3|83|2|85|2|87|2|89|3|92|1|93|2|95|9|122500008960444yesnononoyes3nofurnished



Tercera Parte

1. Configuración inicial del disco:

- Platos.
- Pistas.
- Sector.
- Bloque.

Característica del Disco :

- Capacidad del disco
- Capacidad del bloque
- Capacidad del sector
- Mostrar el árbol de creación del disco

- Cuántas pistas
- Cuantos platos
- Cuantos sectores

Mostrar los criterios para guardar información en los sectores del disco.

REcuerden que el desempeño del disco se mide por:

- Tiempo de transferencia
- Tiempo de acceso
- Capacidad
-



D:\DBPlusPlus\x64\Debug\DB



=== Configuracion del Disco ===

- 1) Usar configuracion por defecto
 - 2) Ingresar configuracion personalizada
 - 3) Usar disco ya existente
- >> |

```
void mostrarArbolDisco() {  
    std::cout << "\n=== Arbol de Creacion del Disco ===\n";  
    for (int i = 0; i < platos; ++i) {  
        std::cout << "Plato " << (i + 1) << "\n";  
  
        for (int s = 0; s < nroSuperficies; ++s) {  
            bool ultimaSuperficie = (s == nroSuperficies - 1);  
  
            std::cout << (ultimaSuperficie ? "|_ " : "|- ")  
                << "Superficie " << (s + 1) << "\n";  
  
            for (int p = 0; p < pistas; ++p) {  
                bool ultimaPista = (p == pistas - 1);  
  
                if (ultimaSuperficie) std::cout << "  ";  
                else std::cout << "| ";  
  
                std::cout << (ultimaPista ? "|_ " : "|- ")  
                    << "Pista " << (p + 1) << "\n";  
  
                for (int sec = 0; sec < sectores; ++sec) {  
                    bool ultimoSector = (sec == sectores - 1);  
  
                    if (ultimaSuperficie) std::cout << "  ";  
                    else std::cout << "| ";  
  
                    if (ultimaPista) std::cout << "  ";  
                    else std::cout << "| ";  
  
                    std::cout << (ultimoSector ? "|_ " : "|- ")  
                        << "Sector " << (sec + 1) << "\n";  
                }  
            }  
        }  
  
        if (i != platos - 1) std::cout << "\n";  
    }  
}
```

=== Arbol de Creacion del Disco ===

Plato 1

```
- Superficie 1  
  - Pista 1  
    - Sector 1  
    - Sector 2  
    - Sector 3  
    - Sector 4  
    - Sector 5  
    - Sector 6  
    - Sector 7  
    - Sector 8  
    - Sector 9  
    - Sector 10  
  - Pista 2  
    - Sector 1  
    - Sector 2  
    - Sector 3  
    - Sector 4  
    - Sector 5  
    - Sector 6  
    - Sector 7  
    - Sector 8  
    - Sector 9  
    - Sector 10  
  - Pista 3
```

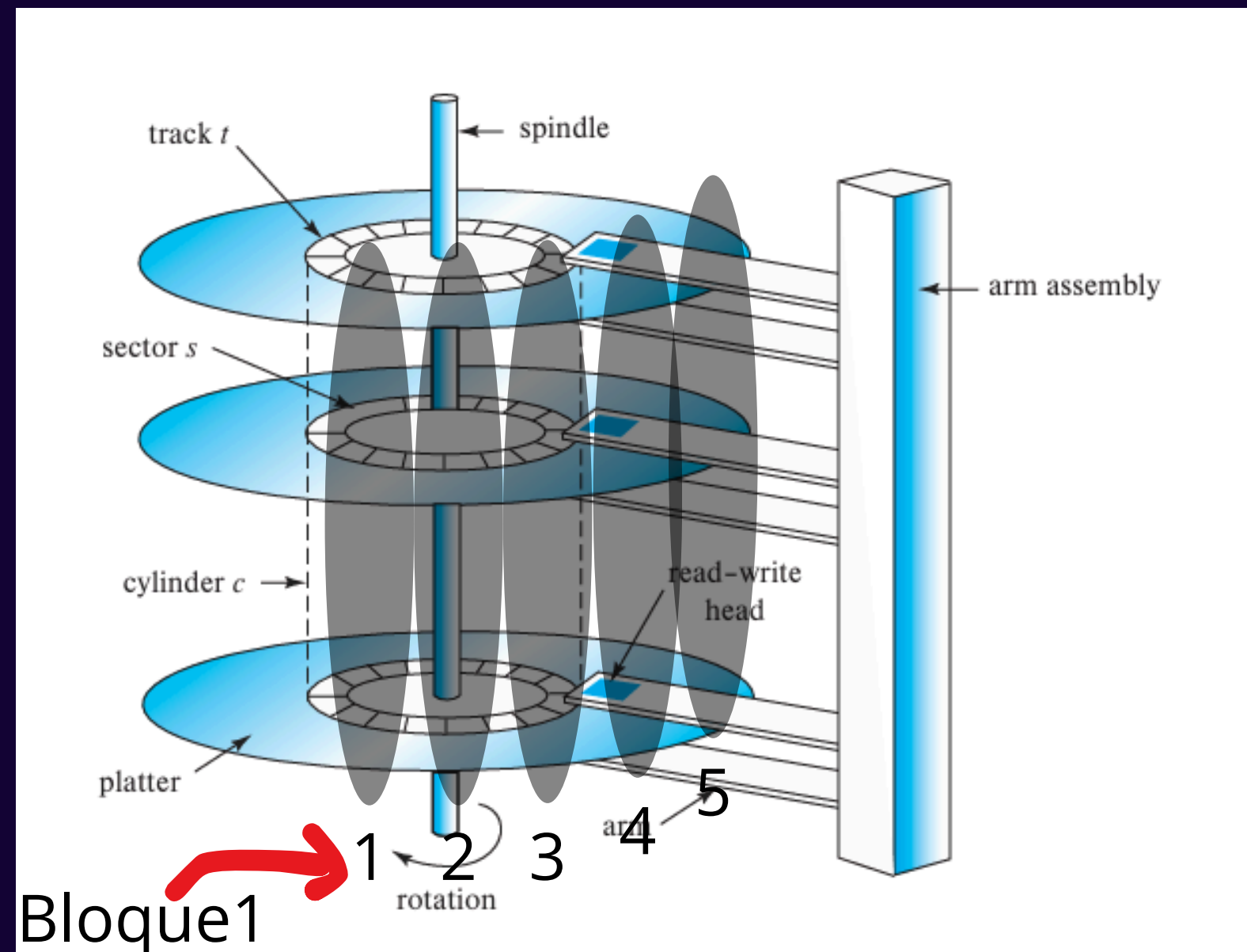
```
Platos: 4
Superficies x plato: 2
Pistas por superficie: 5
Sectores por pista: 10
Tamaño de sector: 800 bytes
Tamaño de bloque: 6400 bytes
Bloques totales (en dirBloques.txt): 50
Bloques por pista: 1
Bloques por plato: 10
Capacidad total: 320000 bytes
Espacio libre (sumado en todos los bloques): 320000 bytes
```

```
{
\n=== Caracteristicas del Disco ===\n";
  Platos: " << platos << "\n";
  Superficies x plato: " << nroSuperficies << "\n";
  Pistas por superficie: " << pistas << "\n";
  Sectores por pista: " << sectores << "\n";
  Tamaño de sector: " << tamSector << " bytes\n";
  Bloques totales: " << nroBloques << "\n";
  Tamaño de bloque: " << tamBloque << " bytes\n";

  bloques por pista y plato
  Pista = (sectores * tamSector) / tamBloque;
  Plato = nroSuperficies * pistas * bloquesPorPista;
```

```
>> 1
Bloque 1 (espacio libre del bloque: |6400):
> Sector: DISCO\Plato1\S1\Pista1\Sector1.txt (espacio libre: 800)
> Sector: DISCO\Plato1\S2\Pista1\Sector1.txt (espacio libre: 800)
> Sector: DISCO\Plato2\S1\Pista1\Sector1.txt (espacio libre: 800)
> Sector: DISCO\Plato2\S2\Pista1\Sector1.txt (espacio libre: 800)
> Sector: DISCO\Plato3\S1\Pista1\Sector1.txt (espacio libre: 800)
> Sector: DISCO\Plato3\S2\Pista1\Sector1.txt (espacio libre: 800)
> Sector: DISCO\Plato4\S1\Pista1\Sector1.txt (espacio libre: 800)
> Sector: DISCO\Plato4\S2\Pista1\Sector1.txt (espacio libre: 800)
> Sector: (espacio libre: @@@@|6400)
```

Estructura del Disco



Silberschatz, A., Korth, H. F., & Sudarshan, S.
Database system concepts.

De forma CONTIGUA
los bloques asignados en el disco duro son adyacentes.

```
| 6400#2#BLOQUE#1#6400
```

```
#_800#1/1/1/1  
#_800#1/2/1/1  
#_800#2/1/1/1  
#_800#2/2/1/1  
#_800#3/1/1/1  
#_800#3/2/1/1  
#_800#4/1/1/1  
#_800#4/2/1/1  
#_@@@@@@@@@|
```


Estructura del Disco

De forma CONTIGUA

los bloques asignados en el disco duro son adyacentes.

```
void crearEstructuraDisco() {
    char rutaTemp[MAX_PATH_LEN];

    for (int p = 1; p <= platos; ++p) {
        // DISCO/Plato<p>
        snprintf(rutaTemp, MAX_PATH_LEN, "DISCO\\Plato%d", p);
        make_dir(rutaTemp);

        for (int s = 1; s <= nroSuperficies; ++s) {
            // DISCO/Plato<p>/S<s>
            snprintf(rutaTemp, MAX_PATH_LEN, "DISCO\\Plato%d\\S%d", p, s);
            make_dir(rutaTemp);

            for (int pi = 1; pi <= pistas; ++pi) {
                // DISCO/Plato<p>/S<s>/Pista<pi>
                snprintf(rutaTemp, MAX_PATH_LEN, "DISCO\\Plato%d\\S%d\\Pista%d", p, s, pi);
                make_dir(rutaTemp);

                for (int se = 1; se <= sectores; ++se) {
                    // Archivo DISCO/Plato<p>/S<s>/Pista<pi>/Sector<se>.txt
                    snprintf(rutaTemp, MAX_PATH_LEN,
                        "DISCO\\Plato%d\\S%d\\Pista%d\\Sector%d.txt",
                        p, s, pi, se);
                    FILE* fsec = fopen(rutaTemp, "w");
                    if (fsec) fclose(fsec);
                }
            }
        }
    }
}
```

| 6400#2#BLOQUE#1#6400

#_800#1/1/1/1
#_800#1/2/1/1
#_800#2/1/1/1
#_800#2/2/1/1
#_800#3/1/1/1
#_800#3/2/1/1
#_800#4/1/1/1
#_800#4/2/1/1
#_@@@@@@@@@|

Estructura del Disco

```
void rutaSectorDesdeCodigo(const char* codigoSector) {  
    int p, s, pi, se;  
    if (sscanf(codigoSector, "%d/%d/%d/%d", &p, &s, &pi, &se) == 4) {  
        snprintf(bufferRuta, MAX_PATH_LEN,  
            "DISCO\\Plato%d\\S%d\\Pista%d\\Sector%d.txt",  
            p, s, pi, se);  
    }  
    else {  
        bufferRuta[0] = '\0';  
    }  
}
```

```
// --- Add your getters here ---
```

| 6400#2#BLOQUE#1#6400

```
#_800#1/1/1/1  
#_800#1/2/1/1  
#_800#2/1/1/1  
#_800#2/2/1/1  
#_800#3/1/1/1  
#_800#3/2/1/1  
#_800#4/1/1/1  
#_800#4/2/1/1  
#_@@@@@@@@@|
```

2. Almacenamiento (REGISTROS):

1. Estructura del Registro :
 - Longitud fija.
 - Longitud variable

Adicionar registros

Mostrar el proceso de creación cuando:

1. Adicionar 1 registro
2. Adicionar N registros de un *.csv
3. Adicionar todo el *.csv

Base de datos (Nro. de registros)

- [Titanic.csv](#)
- [Housing.csv](#)

1. Al adicionar un registro:
Simular para un sector sin suficiente espacio.
2. Al adicionar un registro:
Simular para sectores llenos.


```

static bool adicionarRegistroUnico(const char* registroTxt, const char* relacion, Disco& disco) {
    int registroSize;
    static char regBuf[MAX_BUF];
    static int regLen;

    // 0.1) Debug entrada
    printf("DEBUG: Entrando a adicionarRegistroUnico para relacion='%s', registroTxt='%s'\n",
        relacion, registroTxt);

    // 0.2) Copia segura de registroTxt sin CR/LF
    char registroSinLF[MAX_BUF];
    strncpy(registroSinLF, registroTxt, MAX_BUF - 1);
    registroSinLF[MAX_BUF - 1] = '\0';
    size_t l = strlen(registroSinLF);
    while (l > 0 && (registroSinLF[l - 1] == '\n' || registroSinLF[l - 1] == '\r')) {
        registroSinLF[--l] = '\0';
    }

    // 0.3) Generar RLF (registro de longitud fija)
    if (!crearRLF(registroSinLF, relacion, regBuf, &regLen)) {
        printf("DEBUG: Error en crearRLF. Saliendo.\n");
        return false;
    }
    printf("DEBUG: crearRLF terminó correctamente. regLen = %d\n", regLen);
    // Verificar que no haya saltos de línea en regBuf[0..regLen-1]
    for (int i = 0; i < regLen; ++i) {
        if (regBuf[i] == '\n' || regBuf[i] == '\r') {
            printf("ERROR: detectado salto de línea en regBuf en posición %d\n", i);
        }
    }
}

```

```

return false;
}

printf("DEBUG: crearRLF terminó correctamente. regLen = %d\n", regLen);
// Verificar que no haya saltos de línea en regBuf[0..regLen-1]
for (int i = 0; i < regLen; ++i) {
    if (regBuf[i] == '\n' || regBuf[i] == '\r') {
        printf("ERROR: detectado salto de línea en regBuf en posición %d\n", i);
    }
}

// 0.4) Mostrar contenido de longitudFija.txt (solo debug)
printf(">>> Debug: Leyendo %s para ver su contenido:\n", rutaLongitudFija);
FILE* ftmp = fopen(rutaLongitudFija, "r");
if (ftmp) {
    char buf[MAX_BUF];
    while (fgets(buf, MAX_BUF, ftmp)) {
        printf("    %s", buf);
    }
    fclose(ftmp);
}
else {
    printf("    ERROR: no se pudo abrir %s!\n", rutaLongitudFija);
}

// 0.5) Obtener registroSize
obtenerRegistroSize(relacion, &registroSize);
printf("DEBUG: registroSize calculado = %d\n", registroSize);
if (registroSize <= 0) {
    fprintf(stderr, "ERROR: No se encontró longitud fija para %s\n", relacion);
    return false;
}
}

```

```

// 0.6) Calcular longitud fija máxima de "bloque" en dirBloques.txt
int fixedLen = disco.obtenerLongitudMaximaBloque1();
if (fixedLen <= 0) {
    printf("ERROR: No se pudo obtener longitud fija máxima de bloque.\n");
    return false;
}

printf("DEBUG: fixedLen (longitud de cada bloque) = %d bytes\n", fixedLen);

// 1) Abrir dirBloques.txt en modo lectura y escritura
printf("DEBUG: intentando abrir dirBloques en '%s'\n", rutaDirBloques);
FILE* fdir = fopen(rutaDirBloques, "r+");
if (!fdir) {
    perror("ERROR: No se puede abrir dirBloques.txt");
    return false;
}

printf("DEBUG: dirBloques.txt abierto con éxito.\n");

char lineaBloque[MAX_BUF];
int nroBloque = 0;
bool foundBlock = false;
char codSectorLibre[MAX_STR_LEN] = { 0 };
long posLineaBloque = 0;

int tamRegistro = registroSize;
int espacioLibreBloque = 0;
int tamUtilAntes = 0;
int espacioLibreSectorAntes = 0;

char savedSectores[MAX_BUF] = { 0 }; // Para guardar sectores del bloque elegido

```

```

// 2) Buscar bloque y sector libres (cada bloque ocupa fixedLen bytes total)
printf("DEBUG: Buscando bloque libre para tamaño de registro = %d\n", tamRegistro);
while (true) {
    posLineaBloque = ftell(fdir);

    int lenBloque = disco.leerBloqueConSeparador(fdir, lineaBloque, MAX_BUF);
    if (lenBloque <= 0) {
        printf("DEBUG: leerBloqueConSeparador devolvió %d (fin de bloques)\n", lenBloque);
        break;
    }

    // Saltar bloques vacíos generados por "||"
    if (lenBloque == 2 && lineaBloque[0] == '|' && lineaBloque[1] == '|') {
        continue;
    }

    nroBloque++;
    printf("DEBUG: Leyendo Bloque # %d (lenBloque = %d bytes)\n", nroBloque, lenBloque);

    // 2.1) Extraer espacioLibreBloque sin strtok:
    char tempEspacio[MAX_BUF];
    strncpy(tempEspacio, lineaBloque + 1, MAX_BUF - 1);
    tempEspacio[MAX_BUF - 1] = '\0';

    char* posHash = strchr(tempEspacio, '#');
    if (!posHash) {
        printf("DEBUG: Bloque # %d sin token inicial. Continúa.\n", nroBloque);
        continue;
    }
}

```

```

| 6338#2#BLOQUE#1#6400#_738#1/1/1/1#_800#1/2/1/1#_800#
2/1/1/1#_800#2/2/1/1#_800#3/1/1/1#_800#3/2/1/1#_800#
4/1/1/1#_800#4/2/1/1#_@@@@@@@@@| | 6400#2#BLOQUE#2#6400#_
800#1/1/1/2

```

```

// 2.2) Extraer la lista de sectores en copiaSectores
char copiaSectores[MAX_BUF];
strncpy(copiaSectores, lineaBloque + 1, MAX_BUF - 1);
copiaSectores[MAX_BUF - 1] = '\0';

char* p = strstr(copiaSectores, "#_");
if (!p) {
    printf("DEBUG: Bloque #%d no tiene '#' (no hay lista de sectores). Continúa.\n", nroBloque);
    continue;
}
p += 2;

// 2.3) Iterar cada par "<espLibreSector>#<codSector>#_"
while (*p) {
    char* inicioEspacioSector = p;
    while (*p && *p != '#') p++;
    if (*p != '#') break;
    *p = '\0';
    int espacioLibreSector = atoi(inicioEspacioSector);
    *p = '#';
    p++;

    char* inicioCodSector = p;
    while (*p && *p != '#') p++;
    if (*p != '#') break;
    *p = '\0';
    char sectorCode[MAX_STR_LEN];
    strncpy(sectorCode, inicioCodSector, MAX_STR_LEN - 1);
    sectorCode[MAX_STR_LEN - 1] = '\0';
    *p = '#';

```

```

// 4.3) Copiar cada par "<esp>#<cod>#_", ajustando el sector elegido
{
    char* p2 = strstr(savedSectores, "#_");
    if (p2) {
        p2 += 2; // al inicio del primer "<espLibreSector>"
        while (*p2) {
            int espSec = atoi(p2);
            while (*p2 && *p2 != '#') ++p2;
            if (!*p2) break;
            ++p2;
            char sectorCode2[MAX_STR_LEN] = { 0 };
            int pos2 = 0;
            while (*p2 && *p2 != '#') {
                sectorCode2[pos2++] = *p2++;
            }
            sectorCode2[pos2] = '\0';

            int nuevoEsp = espSec;
            if (strcmp(sectorCode2, codSectorLibre) == 0) {
                nuevoEsp = espSec - tamRegistro;
            }

            int n = snprintf(bufferNueva + ofsN, MAX_BUF - ofsN,
                "%d#%s#_", nuevoEsp, sectorCode2);
            printf("DEBUG: Añadiendo \"%d#%s#_\" => n = %d\n", nuevoEsp, sectorCode2, n);
            ofsN += n;

            char* next2 = strstr(p2, "#_");
            if (!next2) break;
            p2 = next2 + 2;
        }
    }

// 4.4) Poner padding '@' hasta fixedLen-1, y cerrar con '|'
if (ofsN < fixedLen - 1) {
    for (int i = ofsN; i < fixedLen - 1; i++) {
        bufferNueva[i] = '@';
    }
    bufferNueva[fixedLen - 1] = '|';
}

```



```
// 13) No hay hueco: anexar al final
printf("DEBUG: Bloque #%d sin espacio libre, se agregará al final.\n", nroBloque);
// Calcular offsetAppend = raw_header_len + numRegAnt_old * (registroSize+1)
long offsetAppend = (long)raw_header_len + (long)(numRegAnt) * (registroSize + 1);
printf("DEBUG: offsetAppend en Bloque%d = %ld\n", nroBloque, offsetAppend);

// Actualizar cabecera (numRegAnt+1, marcar nuevo bitmap)
numRegAnt++;
if (numRegAnt <= numMaxAnt) {
    bitmapAnt[numRegAnt - 1] = '1';
}

printf("DEBUG: (Anexar) Nuevo numRegAnt = %d, bitmapAnt = '%s'\n",
        numRegAnt, bitmapAnt);

// Reconstruir cabecera EXACTA: espacios hasta raw_header_len-1, luego '/'
rewind(fbloc);
char newHeader3[MAX_BUF];
int ofh3 = 0;
ofh3 += snprintf(newHeader3 + ofh3, MAX_BUF - ofh3, "%d#%d#", numRegAnt, numMaxAnt);
for (int i = 0; i < numMaxAnt && ofh3 < (int)(raw_header_len - 1); i++) {
    newHeader3[ofh3++] = bitmapAnt[i];
}

while (ofh3 < (int)(raw_header_len - 1)) {
    newHeader3[ofh3++] = ' ';
}

newHeader3[raw_header_len - 1] = '/';
newHeader3[raw_header_len] = '\0';
```

```
0000000000000000000000000000000000000000000000000000000000000000  
0000000000000000000000000000000000000000000000000000000/1790000  
@#4000@#3#1#2#yes#no@#no@#no@#no@#0  
#no@#unfurnished@@@# |
```



```
// 13) No hay hueco: anexar al final
printf("DEBUG: Bloque #%d sin espacio libre, se agregará al final.\n", nroBloque);
// Calcular offsetAppend = raw_header_len + numRegAnt_old * (registroSize+1)
long offsetAppend = (long)raw_header_len + (long)(numRegAnt) * (registroSize + 1);
printf("DEBUG: offsetAppend en Bloque%d = %ld\n", nroBloque, offsetAppend);

// Actualizar cabecera (numRegAnt+1, marcar nuevo bitmap)
numRegAnt++;
if (numRegAnt <= numMaxAnt) {
    bitmapAnt[numRegAnt - 1] = '1';
}

printf("DEBUG: (Anexar) Nuevo numRegAnt = %d, bitmapAnt = '%s'\n",
        numRegAnt, bitmapAnt);

// Reconstruir cabecera EXACTA: espacios hasta raw_header_len-1, luego '/'
rewind(fbloc);
char newHeader3[MAX_BUF];
int ofh3 = 0;
ofh3 += snprintf(newHeader3 + ofh3, MAX_BUF - ofh3, "%d#%d#", numRegAnt, numMaxAnt);
for (int i = 0; i < numMaxAnt && ofh3 < (int)(raw_header_len - 1); i++) {
    newHeader3[ofh3++] = bitmapAnt[i];
}
while (ofh3 < (int)(raw_header_len - 1)) {
    newHeader3[ofh3++] = ' ';
}
newHeader3[raw_header_len - 1] = '/';
newHeader3[raw_header_len] = '\0';
```

```
0000000000000000000000000000000000000000000000000000000000000000  
0000000000000000000000000000000000000000000000000000000/1790000  
@#4000@#3#1#2#yes#no@#no@#no@#no@#0  
#no@#unfurnished@@@# |
```

```

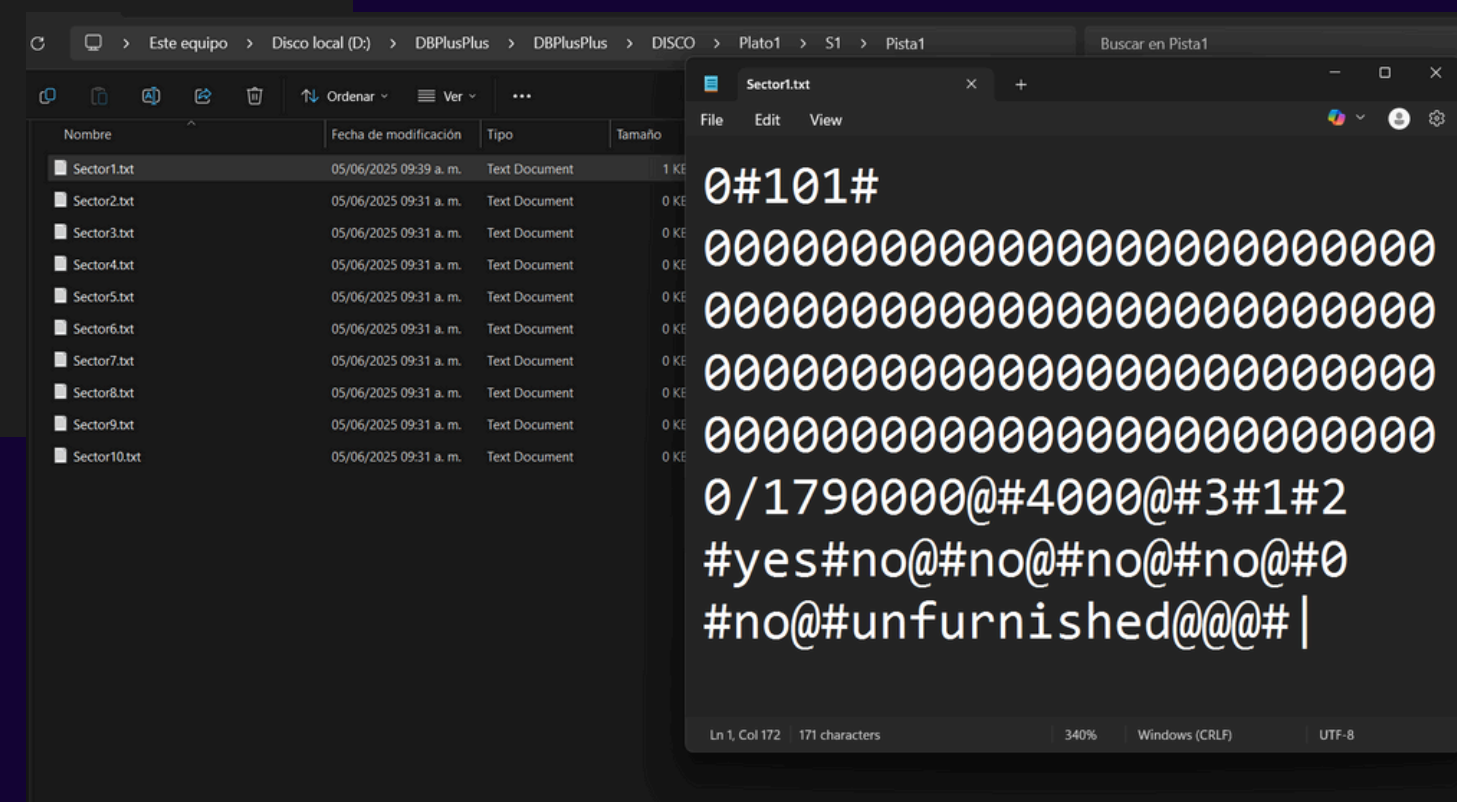
// Insertar regBuf + '|' en offsetAppend
if (fseek(fbloc, offsetAppend, SEEK_SET) != 0) {
    fprintf(stderr, "ERROR: fseek falló al offsetAppend\n");
    fclose(fbloc);
    return false;
}

size_t escritosAppend = fwrite(regBuf, 1, regLen, fbloc);
fputc('|', fbloc);
fflush(fbloc);
if (escritosAppend != (size_t)regLen) {
    printf("ERROR: Se escribieron %zu bytes de RLF anexado, esperados %d\n", escritosAppend, regLen);
}
else {
    printf("DEBUG: Registro de %d bytes anexado correctamente (+ '|').\n", regLen);
}

fclose(fbloc);
// 14) Volcar bloque anexado a sectores
printf("DEBUG: Llamando a volcarBloqueASectores para bloque #%d (anexar)\n", nroBloque);
disco.volcarBloqueASectores(nroBloque);

printf("DEBUG: adicionarRegistroUnico (anexar) finalizado para Bloque #%d\n", nroBloque);
return true;
}

```



```

// Insertar regBuf + '|' en offsetAppend
if (fseek(fbloc, offsetAppend, SEEK_SET) != 0) {
    fprintf(stderr, "ERROR: fseek falló al offsetAppend\n");
    fclose(fbloc);
    return false;
}

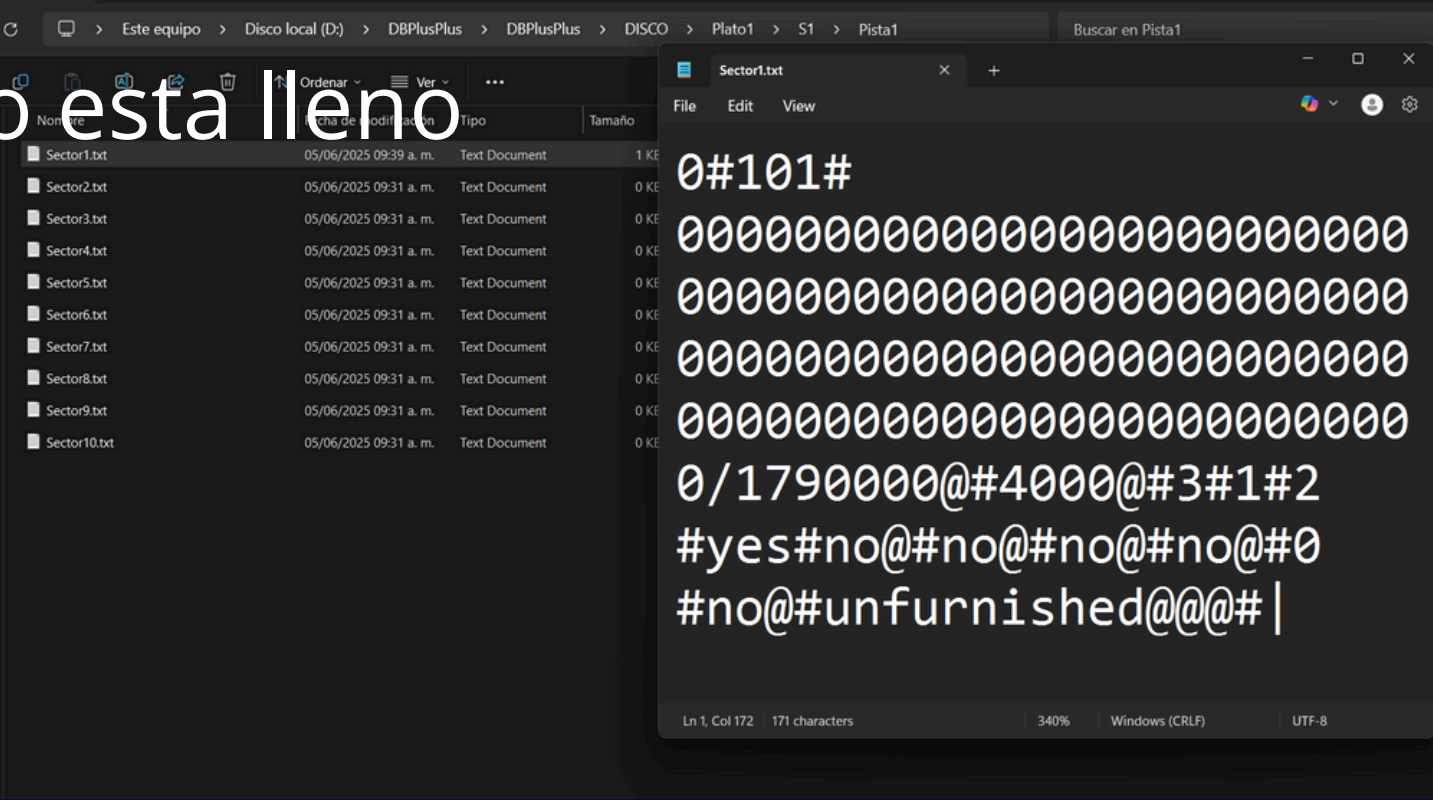
size_t escritosAppend = fwrite(regBuf, 1, regLen, fbloc);
fputc('|', fbloc);
fflush(fbloc);
if (escritosAppend != (size_t)regLen) {
    printf("ERROR: Se escribieron %zu bytes de RLF anexado, esperados %d\n", escritosAppend, regLen);
}
else {
    printf("DEBUG: Registro de %d bytes anexado correctamente (+ '|').\n", regLen);
}

fclose(fbloc);
// 14) Volcar bloque anexado a sectores
printf("DEBUG: Llamando a volcarBloqueASectores para bloque # %d (anexar)\n", nroBloque);
disco.volcarBloqueASectores(nroBloque);

printf("DEBUG: adicionarRegistroUnico (anexar) finalizado para Bloque # %d\n", nroBloque);
return true;
}

```

Simular mensajes cuando esta lleno



Eliminar Registros Fija

```

else if (opt == "5") {
    cout << "¿Tipo de eliminacion?\n";
    cout << "1) Longitud variable\n";
    cout << "2) Longitud fija (bitmap)\n";
    cout << ">> ";
    int opcion;
    cin >> opcion;
    cin.ignore(1, '\n');

    if(opcion ==2){
        if (eliminarRegistro("housing", 2, miDisco)) {
            cout << "Eliminado correctamente\n";
            cout << "Entrando a Eliminar Registro \n" << std::endl;
        }
    }
    if(opcion==1){
        bool ok = eliminar_registro_variable(1,2, miDisco.getTamBloque(),miDisco);
        assert(ok);
    }
    else {
        cout << "No se pudo eliminar el registro \n";
    }
}
}

```

[illegible]


```

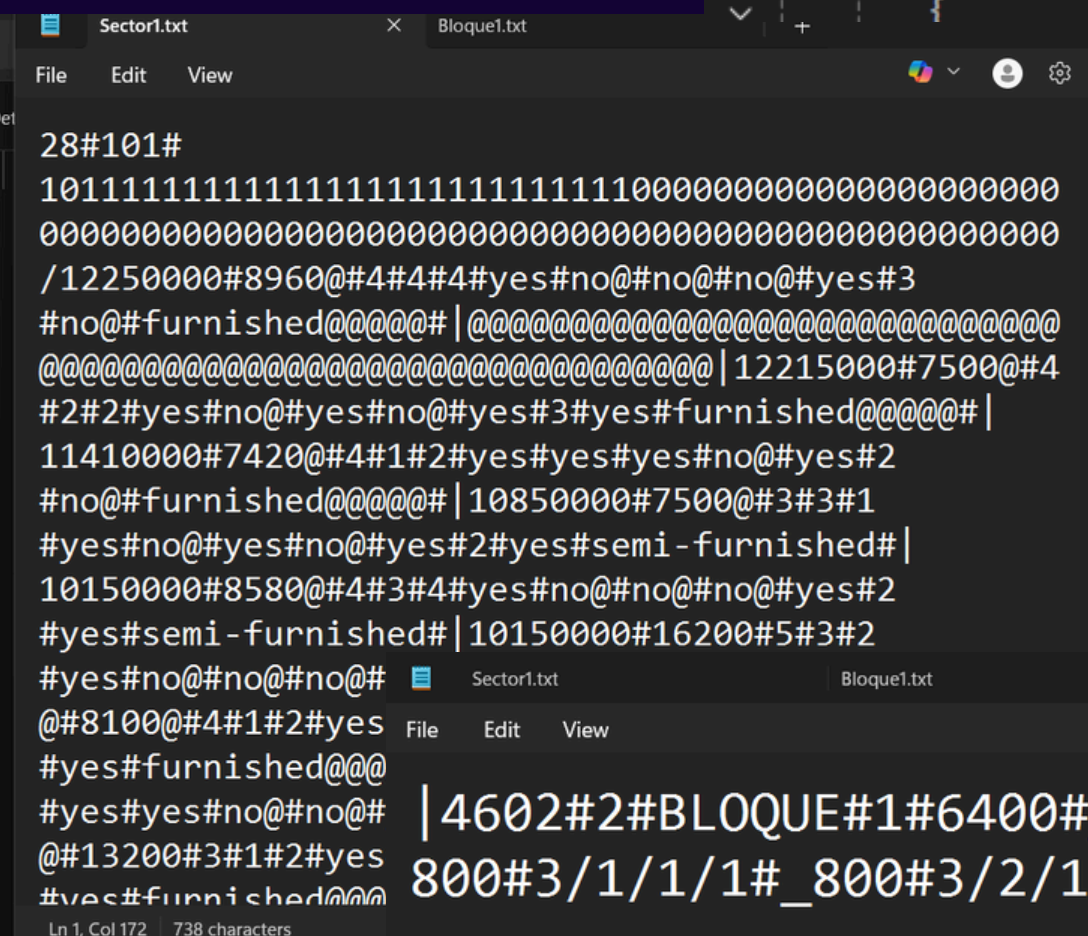
else if (opt == "5") {
    cout << "¿Tipo de eliminacion?\n";
    cout << "1) Longitud variable\n";
    cout << "2) Longitud fija (bitmap)\n";
    cout << ">> ";
    int opcion;
    cin >> opcion;
    cin.ignore(1, '\n');

    if(opcion ==2){
        if (eliminarRegistro("housing", 2, miDisco)) {
            cout << "Eliminado correctamente\n";
            cout << "Entrando a Eliminar Registro \n" << std::endl;
        }
    }
    if(opcion==1){
        bool ok = eliminar_registro_variable(1,2, miDisco.getTamBloque(),miDisco);
        assert(ok);
    }
    else {
        cout << "No se pudo eliminar el registro \n";
    }
}
}

```

[illegible]





```
|4602#2#BLOQUE#1#6400#_118#1/1/1/1#_56#1/2/1/1#_428#2/1/1/1#_800#2/2/1/1#_800#3/1/1/1#_800#3/2/1/1#_800#4/1/1/1#_800#4/2/1/1#_@@@@@@@@@@|
```

```
// 2.1) Leer cabecera para extraer raw_header_len, numRegAnt, numMaxAnt y bitmap[]
```

```
char cabTmp[MAX_BUF];
```

```
int numRegAnt = 0;
```

```
static char bitmap[MAX_BUF];
```

```
{
```

```
    size_t pos = 0;
```

```
    int ch;
```

```
    rewind(fbloc);
```

```
    while ((ch = fgetc(fbloc)) != EOF) {
```

```
        pos++;
```

```
        if (ch == '/') break;
```

```
        if (pos >= MAX_BUF - 1) break;
```

```
    }
```

```
    if (ch != '/') {
```

```
        fprintf(stderr, "ERROR: Cabecera corrupta o falta '/' en Bloque%d\n", bloqueN);
```

```
        fclose(fbloc);
```

```
        return false;
```

```
    }
```

```
    raw_header_len = (pos > MAX_BUF - 1 ? MAX_BUF - 1 : pos);
```

```
}
```

```
rewind(fbloc);
```

```
fread(cabTmp, 1, raw_header_len, fbloc);
```

```
cabTmp[raw_header_len] = '\0';
```

```
// 2.2) Verificar que el slot idxLocal esté ocupado
```

```
if (idxLocal < 0 || idxLocal >= numMaxAnt) {
```

```
    fprintf(stderr, "ERROR: Índice local %d fuera de rango en Bloque%d\n", idxLocal, bloqueN);
```

```
    fclose(fbloc);
```

```
    return false;
```

```
}
```

```
if (bitmap[idxLocal] == '0') {
```

```
    printf("DEBUG: El registro local %d ya estaba eliminado (bitmap=0)\n", idxLocal + 1);
```

```
    fclose(fbloc);
```

```
    return false;
```

```
}
```

```
// 2.3) Actualizar bitmap y numRegAnt (numMaxAnt permanece igual)
```

```
bitmap[idxLocal] = '0';
```

```
numRegAnt--;
```

```
printf("DEBUG: bitmap[%d] cambiado a '0', numRegAnt ahora = %d\n", idxLocal, numRegAnt);
```

```
// 2.4) Reconstruir la cabecera EXACTA de longitud raw_header_len
```

```
{
```

```
    char newHeader[MAX_BUF];
```

```
    int ofh = 0;
```

```
    ofh += snprintf(newHeader + ofh, MAX_BUF - ofh, "%d#%d#", numRegAnt, numMaxAnt);
```

```
    for (int i = 0; i < numMaxAnt - ofh + (int)(raw_header_len - 1); i++)
```



```
// 2.6) Reemplazar los bytes del registro con '@'
if (fseek(fbloc, offsetRegistro, SEEK_SET) != 0) {
    fprintf(stderr, "ERROR: fseek falló al offsetRegistro\n");
    fclose(fbloc);
    return false;
}

char* relleno = (char*)malloc(registroSize);
memset(relleno, '@', registroSize);
size_t escritos = fwrite(relleno, 1, registroSize, fbloc);
free(relleno);
if ((int)escritos != registroSize) {
    fprintf(stderr, "ERROR: Se escribieron %zu bytes, esperados %d\n", escritos, registroSize);
    fclose(fbloc);
    return false;
}

fflush(fbloc);
printf("DEBUG: Registro de %d bytes en Bloque%d reemplazado por '@'\n", registroSize, bloqueN);

// 2.7) Calcular sector físico donde residía ese registro
int tarSector = disco.getTarSector();
```

[illegible]

```
// 2.6) Reemplazar los bytes del registro con '@'
if (fseek(fbloc, offsetRegistro, SEEK_SET) != 0) {
    fprintf(stderr, "ERROR: fseek falló al offsetRegistro\n");
    fclose(fbloc);
    return false;
}

char* relleno = (char*)malloc(registroSize);
memset(relleno, '@', registroSize);
size_t escritos = fwrite(relleno, 1, registroSize, fbloc);
free(relleno);
if ((int)escritos != registroSize) {
    fprintf(stderr, "ERROR: Se escribieron %zu bytes, esperados %d\n", escritos, registroSize);
    fclose(fbloc);
    return false;
}

fflush(fbloc);
printf("DEBUG: Registro de %d bytes en Bloque%d reemplazado por '@'\n", registroSize, bloqueN);

// 2.7) Calcular sector físico donde residía ese registro
int tarSector = disco.getTarSector();
```

[illegible]

10) Salir

>> 2

Ingrese registro: 1790000#4000#3#1#2#yes#no#no#no#no#0#no#unfurnished

7 Tipo de inserción?

1) Longitud variable

2) Longitud fija (bitmap)

>> 2

DEBUC: Entenda o adicionalDebitControlInício por 307 de 307 | [home](#) | [navegação](#) | [navegação](#)

29#101#

```
111111111111111111111111111000000000000000000000000000000000000000000000000000000000000
```

[illegible]

```
#no@#furnished@@@#@# | 1790000@#4000@#3#1#2#yes#no@#no@#no@#no@#0
```

```
#no@#unfurnished@@@# | 12215000#7500@#4#2#2#yes#no@#yes#no@#yes#3
```

```
#yes#furnished@@@@# | 11410000#7420@#4#1#2#yes#yes#yes#no@#yes#2
```

```
#no@#furnished@@@#@# | 10850000#7500@#3#3#1#yes#no@#yes#no@#yes#2#yes#semi-
```

furnished#|10150000#8580@#4#3#4#yes#no@#no@#no@#yes#2#yes#semi-furnished#|

10150000#16200#5#3#2#yes#no@#no@#no@#no@#0#no@#unfurnished@@@#|9870000@#8100@#

4#1#2#yes#yes#yes#no@#yes#2#yes#furnished@@@@@# | 9800000@#5750@#3#2#4

```
#yes#yes#no@#no@#yes#1#yes#unfurnished@@@#|9800000@#13200#3#1#2
```

Modificar Registro

```
else if (opt == "6") {
    cout << "Tipo de Modificacion?\n";
    cout << "1) Longitud variable\n";
    cout << "2) Longitud fija\n";
    cout << ">> ";

    const char* nuevoReg = "1790000#9#3#1#2#yes#no#no#no#0#no#semi-furnished";

    if (modificarRegistro("housing", 4, nuevoReg, miDisco)) {
        cout << "Modificado correctamente\n";
    }
    if(modify_registro_variable(125, 2, "titanic", "Sex", "Male",miDisco)){
    }
}
```

29#101#

[illegible]

Modificar Registro

```
else if (opt == "6") {
    cout << "Tipo de Modificacion?\n";
    cout << "1) Longitud variable\n";
    cout << "2) Longitud fija\n";
    cout << ">> ";

    const char* nuevoReg = "1790000#9#3#1#2#yes#no#no#no#0#no#semi-furnished";

    if (modificarRegistro("housing", 4, nuevoReg, miDisco)) {
        cout << "Modificado correctamente\n";
    }
    if(modify_registro_variable(125, 2, "titanic", "Sex", "Male",miDisco)){
    }
}
```

29#101#

```
11111111111111111111111110000000000000000000000000000000  
000000000000000000000000/12250000#8960@#4#4#4#yes#no@#no@#no@#yes#3  
#no@#furnished@@@@#@|1790000@#4000@#3#1#2#yes#no@#no@#no@#no@#0  
#no@#unfurnished@@@@#@|12215000#7500@#4#2#2#yes#no@#yes#no@#yes#3  
#yes#furnished@@@@#@|1790000@#9@@@@#@#3#1#2#yes#no@#no@#no@#no@#0#no@s  
furnished#|10850000#7500@#3#3#1#yes#no@#yes#no@#yes#2#yes#semi-furnis  
10150000#8580@#4#3#4#yes#no@#no@#no@#yes#2#yes#semi-furnished#|10150  
5#3#2#yes#no@#no@#no@#no@#0#no@#unfurnished@@@@#@|9870000@#8100@#4#1#2  
#yes#yes#yes#no@#yes#2#yes#furnished@@@@#@|9800000@#5750@#3#2#4
```


Modificar Registro

```
static bool modificarRegistro(const char* relacion, int posicion, const char* nuevoRegistroTxt, Disco disco) {
    printf("DEBUG: Entrando a modificarRegistro para relación='%s', posición=%d\n", relacion, posicion);

    // 1) Determinar en qué bloque y qué índice local corresponde a la posición global.
    int bloqueN = 0;
    int rem = posicion;
    int numMaxAnt = 0;
    size_t raw_header_len = 0;
    char rutaBloque[MAX_PATH_LEN];

    while (true) {
        bloqueN++;
        snprintf(rutaBloque, sizeof(rutaBloque),
            "DISCO\\BLOQUES\\Bloque%d.txt", bloqueN);
        FILE* ftest = fopen(rutaBloque, "rb");
        if (!ftest) {
            printf("ERROR: No existe Bloque%d.txt; posición %d fuera de rango.\n", bloqueN, posicion);
            return false;
        }
        // Leer solo hasta '/' para extraer numMaxAnt
        size_t headerLen = 0;
        int ch;
        rewind(ftest);
        while ((ch = fgetc(ftest)) != EOF) {
            headerLen++;
            if (ch == '/') break;
            if (headerLen >= MAX_BUF - 1) break;
        }
        if (ch != '/') {
            fprintf(stderr, "ERROR: Cabecera corrupta en Bloque%d\n", bloqueN);
            fclose(ftest);
            return false;
        }
    }
```

```
        rem -= numMaxAnt;
    }
    int idxLocal = rem - 1; // índice 0-based dentro del bloque
    printf("DEBUG: La posición %d cae en Bloque%d, idxLocal=%d\n", posicion, bloqueN, idxLocal);

    // 2) Eliminar el registro antiguo en esa posición global
    if (!eliminarRegistro(relacion, posicion, disco)) {
        printf("ERROR: eliminarRegistro falló para posición %d\n", posicion);
        return false;
    }
    printf("DEBUG: eliminarRegistro completado (Bloque%d, idxLocal=%d)\n", bloqueN, idxLocal);

    // 3) Generar RLF del nuevo registro
    char regBuf[MAX_BUF];
    int regLen = 0;
    if (!crearRLF(nuevoRegistroTxt, relacion, regBuf, &regLen)) {
        printf("ERROR: crearRLF falló para modificación.\n");
        return false;
    }
    int registroSize = 0;
    obtenerRegistroSize(relacion, &registroSize);
    if (registroSize <= 0) {
        fprintf(stderr, "ERROR: No se encontró registroSize para '%s'\n", relacion);
        return false;
    }

    // 4) Reabrir BloqueN.txt para insertar en idxLocal
    FILE* fbloc = fopen(rutaBloque, "r+b");
    if (!fbloc) {
        perror("ERROR: No se pudo reabrir BloqueN.txt");
        return false;
    }

    // 4.1) Leer cabecera completa hasta '/'
    char cabTmp[MAX_BUF];
    rewind(fbloc);
```

Modificar Registro

```

// 4.1) Marcar bitmap[idxLocal]=1 y numRegAnt++
bitmap[idxLocal] = '1';
numRegAnt++;
printf("DEBUG: (modificar) bitmap actualizado: '%s'\n", numMax, bitmap);

// Reconstruir cabecera exacta al mismo largo raw_header_len
char newHead[MAX_BUF];
int ofh = 0;
ofh += snprintf(newHead + ofh, MAX_BUF - ofh, "%d#%d#", numRegAnt, numMax);
for (int i = 0; i < numMax && ofh < (int)(raw_header_len - 1); i++) {
    newHead[ofh++] = bitmap[i];
}
newHead[ofh++] = '/';
while (ofh < (int)raw_header_len) {
    newHead[ofh++] = ' ';
}
newHead[ofh] = '\0';

rewind(fbloc);
fwrite(newHead, 1, raw_header_len, fbloc);
fflush(fbloc);
printf("DEBUG: Cabecera de Bloque%d reescrita (modificar): '%s'\n",
       bloqueN, (int)raw_header_len, newHead);

// 4.3) Calcular offset fijo y escribir regBuf + '|'
long offsetRegistro = (long)raw_header_len + (long)idxLocal * (registroSize + 1);
if (fseek(fbloc, offsetRegistro, SEEK_SET) != 0) {
    fprintf(stderr, "ERROR: fseek falló al offsetRegistro %ld\n", offsetRegistro);
    fclose(fbloc);
    return false;
}
fwrite(regBuf, 1, registroSize, fbloc);
fclose(fbloc);

```

[illegible]

Modificar Registro

[illegible]

```

// 4.2) Marcar bitmap[idxLocal]='1' y numRegAnt++
bitmap[idxLocal] = '1';
numRegAnt++;
printf("DEBUG: (modificar) bitmap actualizado: '%.*s'\n", numMax, bitmap);

// Reconstruir cabecera exacta al mismo largo raw_header_len
char newHead[MAX_BUF];
int ofh = 0;
ofh += snprintf(newHead + ofh, MAX_BUF - ofh, "%d#%d#", numRegAnt, numMax);
for (int i = 0; i < numMax && ofh < (int)raw_header_len - 1; i++) {
    newHead[ofh++] = bitmap[i];
}
newHead[ofh++] = '/';
while (ofh < (int)raw_header_len) {
    newHead[ofh++] = ' ';
}
newHead[ofh] = '\0';

rewind(fbloc);
fwrite(newHead, 1, raw_header_len, fbloc);
fflush(fbloc);
printf("DEBUG: Cabecera de Bloque%d reescrita (modificar): '%.*s'\n",
       bloqueN, (int)raw_header_len, newHead);

// 4.3) Calcular offset fijo y escribir regBuf + '|'
long offsetRegistro = (long)raw_header_len + (long)idxLocal * (registroSize + 1);
if (fseek(fbloc, offsetRegistro, SEEK_SET) != 0) {
    fprintf(stderr, "ERROR: fseek falló al offsetRegistro %ld\n", offsetRegistro);
    fclose(fbloc);
    return false;
}

fwrite(regBuf, 1, registroSize, fbloc);
fclose(fbloc);

```

[illegible]

Modificar Registro

[illegible]

```

// 4.2) Marcar bitmap[idxLocal]='1' y numRegAnt++
bitmap[idxLocal] = '1';
numRegAnt++;
printf("DEBUG: (modificar) bitmap actualizado: '%s'\n", numMax, bitmap);

// Reconstruir cabecera exacta al mismo largo raw_header_len
char newHead[MAX_BUF];
int ofh = 0;
ofh += snprintf(newHead + ofh, MAX_BUF - ofh, "%d#%d#", numRegAnt, numMax);
for (int i = 0; i < numMax && ofh < (int)(raw_header_len - 1); i++) {
    newHead[ofh++] = bitmap[i];
}
newHead[ofh++] = '/';
while (ofh < (int)raw_header_len) {
    newHead[ofh++] = ' ';
}
newHead[ofh] = '\0';

rewind(fbloc);
fwrite(newHead, 1, raw_header_len, fbloc);
fflush(fbloc);
printf("DEBUG: Cabecera de Bloque%d reescrita (modificar): '%s'\n",
       bloqueN, (int)raw_header_len, newHead);

// 4.3) Calcular offset fijo y escribir regBuf + '|'
long offsetRegistro = (long)raw_header_len + (long)idxLocal * (registroSize + 1);
if (fseek(fbloc, offsetRegistro, SEEK_SET) != 0) {
    fprintf(stderr, "ERROR: fseek falló al offsetRegistro %ld\n", offsetRegistro);
    fclose(fbloc);
    return false;
}
fwrite(regBuf, 1, registroSize, fbloc);
fclose(fbloc);

```

[illegible]

Modificar Registro

```
Sector1.txt Bloque1.txt dirBloques.txt
File Edit View
|4602#2#BLOQUE#1#6400#_118#1/1/1/1#_56#1/2/1/1#_428#2/1/1/1#_800#2/2/1/1#_800#
3/1/1/1#_800#3/2/1/1#_800#4/1/1/1#_800#4/2/1/1#_@@@@@@@@@@|6400#2#BLOQUE#2#
6400# 800#1/1/1/2# 800#1/2/1/2# 800#2/1/1/2# 800#2/2/1/2# 800#3/1/1/2# 800#
```

```
plus (Global Scope) mod
// 4) Calcular offset registro y escribir registro
long offsetRegistro = (long)raw_header_len + (long)idxLocal * (registroSize + 1);
if (fseek(fbloc, offsetRegistro, SEEK_SET) != 0) {
    fprintf(stderr, "ERROR: fseek falló al offsetRegistro %ld\n", offsetRegistro);
    fclose(fbloc);
    return false;
}
fwrite(regBuf, 1, registroSize, fbloc);
fputc('|', fbloc);
fflush(fbloc);
fclose(fbloc);
printf("DEBUG: Nuevo RLF escrito en Bloque%d, idxLocal=%d\n", bloqueN, idxLocal);

// 5) Volcar BloqueN a sectores para que el cambio se refleje en disco físico
printf("DEBUG: Llamando a volcarBloqueASectores (modificar) para Bloque%d\n", bloqueN);
disco.volcarBloqueASectores(bloqueN);

// 6) No es necesario ajustar dirBloques.txt de nuevo
return true;
}
```


Numero de Bloque que desea consultar: 1

Ingresa opcion

1) Con Espacios Libres

2) Rutas Solamente

>> 1

Bloque 1 (espacio libre del bloque: |4602):

- > Sector: DISCO\Plato1\S1\Pista1\Sector1.txt (espacio libre: 118)
- > Sector: DISCO\Plato1\S2\Pista1\Sector1.txt (espacio libre: 56)
- > Sector: DISCO\Plato2\S1\Pista1\Sector1.txt (espacio libre: 428)
- > Sector: DISCO\Plato2\S2\Pista1\Sector1.txt (espacio libre: 800)
- > Sector: DISCO\Plato3\S1\Pista1\Sector1.txt (espacio libre: 800)
- > Sector: DISCO\Plato3\S2\Pista1\Sector1.txt (espacio libre: 800)
- > Sector: DISCO\Plato4\S1\Pista1\Sector1.txt (espacio libre: 800)
- > Sector: DISCO\Plato4\S2\Pista1\Sector1.txt (espacio libre: 800)
- > Sector: (espacio libre: @@@@|6400)

Capacidad máxima del registro

Característica del Disco :

- Capacidad del disco
- Capacidad libre
- Capacidad ocupada
- Cuantos sectores están ocupados
- Esos sectores ocupados donde están ubicados En qué plato, superficie, pista y sector están los registros del bloque? -

Eliminar registros

- Explicar cómo se eliminan registros
- Eliminar un registro
- Mostrar cómo manejan la eliminación de registros a nivel de cabecera
- En qué sector, pista, plato, están eliminando los registros.

```
=== Características del Disco (dinámico desde dirBloques.txt) ===
Platos: 4
Superficies x plato: 2
Pistas por superficie: 5
Sectores por pista: 10
Tamaño de sector: 800 bytes
Tamaño de bloque: 6400 bytes
Bloques totales (en dirBloques.txt): 50
Bloques por pista: 1
Bloques por plato: 10
Capacidad total: 320000 bytes
Espacio libre (sumado en todos los bloques): 318202 bytes
=====
=== ----- ===
```

=== Características del Disco (dinámico desde dirBloques.txt) ===

Platos: 4

Superficies x plato: 2

Pistas por superficie: 5

Sectores por pista: 10

Tamaño de sector: 800 bytes

Tamaño de bloque: 6400 bytes

Bloques totales (en dirBloques.txt): 50

Bloques por pista: 1

Bloques por plato: 10

Capacidad total: 320000 bytes

Espacio libre (sumado en todos los bloques): 320000 bytes

=====

```
: El registro estaba en sector fisico '1/1/1/1' ? Plato 1, Pista 1, Sector 1
```

[illegible]

_: El registro estaba en sector físico '1/1/1/1' ? Plato 1, Pista 1, Sector 1

3. BLOQUES

Explicar

Mostrar contenido de un bloque.

Mostrar cabecera del bloque

Capacidad de bloque

Mostrar bloques ocupados y bloques vacíos

- Mostrar capacidad de cada bloque

SELECT * FROM

WHERE

Cabeceras

Variable

```
1 2|6190|6295|105|6191|104|NULNULNULNULNULNULNULNULNULNULNULNUL
```

Fija

[illegible]

3. BLOQUES

Explicar

Mostrar contenido de un bloque.

Mostrar cabecera del bloque

Capacidad de bloque

Mostrar bloques ocupados y bloques vacios

- Mostrar capacidad de cada bloque

SELECT * FROM

WHERE

```
10) Salir
```

```
>> 0
```

```
Numero de Bloque que desea consultar: 1
```

```
Ingresa opcion
```

```
1) Con Espacios Libres
```

```
2) Rutas Solamente
```

```
>> 1
```

```
Bloque 1 (espacio libre del bloque: |448):
```

```
> Sector: DISCO\Plato1\S1\Pista1\Sector1.txt (espacio libre: 56)
```

```
> Sector: DISCO\Plato1\S2\Pista1\Sector1.txt (espacio libre: 56)
```

```
> Sector: DISCO\Plato2\S1\Pista1\Sector1.txt (espacio libre: 56)
```

```
> Sector: DISCO\Plato2\S2\Pista1\Sector1.txt (espacio libre: 56)
```

```
> Sector: DISCO\Plato3\S1\Pista1\Sector1.txt (espacio libre: 56)
```

```
> Sector: DISCO\Plato3\S2\Pista1\Sector1.txt (espacio libre: 56)
```

```
> Sector: DISCO\Plato4\S1\Pista1\Sector1.txt (espacio libre: 56)
```

```
> Sector: DISCO\Plato4\S2\Pista1\Sector1.txt (espacio libre: 56)
```




Referencias

- Silberschatz, A., Korth, H. F., & Sudarshan, S. Database system concepts.

